A Project report on

ENHANCING EMERGENCY CARE THROUGH MACHINE LEARNING BASED ADMISSION PRIORITIZATION

Submitted in partial fulfillment of the requirement

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING(AI)

By

K.REVATHY	**22701A3135**
D.KARTHIK	**22701A3122**
P.USHA SREE	**22701A3144**
M.SUNIL	**22701A3142**
S.MANOJ KUMAR	**23705A3103**

Under the esteemed guidance of

Dr.C.V.LAKSHMI NARAYANA,M.Tech,Ph.D.,

Assistant Professor

Department of AI&ML, AITS.

Submitted to

Department of Computer Science and Engineering(AI)

Annamacharya Institute of Technology and Sciences

(An Autonomous Institution)

(Approved by AICTE, New-Delhi and affiliated to J.N.T.U, Anantapur)

(Accredited by NBA &NAAC A+)

New Boyanapalli, Rajampet, Kadapa (Dt), A.P-516 126

2025-2026

Department of Computer Science and Engineering(AI)

Annamacharya Institute of Technology and Sciences

(An Autonomous Institution)

(Approved by AICTE, New-Delhi and affiliated to J.N.T.U, Anantapur)

(Accredited by NBA & NAAC A+)

New Boyanapalli, Rajampet , Kadapa (Dt), A.P-516 126

CERTIFICATE

This is to certify that the project report entitled **ENHANCING EMERGENCY CARE THROUGH MACHINE LEARNING BASED ADMISSION PRIORITIZATION** is submitted by

K.REVATHY	**22701A3135**
D.KARTHIK	**22701A3122**
P.USHA SREE	**22701A3144**
M.SUNIL	**22701A3142**
S.MANOJ KUMAR	**23705A3103**

in partial fulfillment of the requirements for the award of Degree of **Bachelor of Technology** in "**Computer Science and Engineering(AI)**" for the academic year 2025-26.

Signature of Guide

Dr.C.V.Lakshmi Narayana, M.Tech, Ph.D.,
Assistant Professor,Dept. of AI&ML
AI&ML AITS,Rajampet.

Signature of HOD

Dr. N.Penchalaiah , M.Tech,Ph.D.,
Associate Professor & HoD, Dept. of AI&ML
AITS, Rajampet.

Department of Computer Science and Engineering(AI)

Annamacharya Institute of Technology and Sciences

(An Autonomous Institution)

(Approved by AICTE, New-Delhi and affiliated to J.N.T.U, Anantapur)

(Accredited by NBA & NAAC A+)

New Boyanapalli, Rajampet, Kadapa (Dt), A.P-516 126

CERTIFICATE

This is to certify that the project report entitled **ENHANCING EMERGENCY CARE THROUGH MACHINE LEARNING BASED ADMISSION PRIORITIZATION** is submitted by

K.REVATHY	**22701A3135**
D.KARTHIK	**22701A3122**
P.USHA SREE	**22701A3144**
M.SUNIL	**22701A3142**
S.MANOJ KUMAR	**23705A3103**

in partial fulfillment of the requirements for the award of Degree of **Bachelor of Technology** in **"Computer Science and Engineering(AI)"** is a record of bonafide work carried out by her during the academic year 2025-26.

Project viva-voce held on : _____

Internal Examiner External Examiner

Department of Computer Science and Engineering(AI)

Annamacharya Institute of Technology and Sciences

(An Autonomous Institution)

(Approved by AICTE, New-Delhi and affiliated to J.N.T.U, Anantapur)

(Accredited by NBA & NAAC A+)

New Boyanapalli, Rajampet, Kadapa (Dt), A.P-516 126

ANTI-PLAGIARISM CERTIFICATE

This is to certify that the project report entitled **ENHANCING EMERGENCY CARE THROUGH MACHINE LEARNING BASED ADMISSION PRIORITIZATION** is submitted by

K.REVATHY	**22701A3135**
D.KARTHIK	**22701A3122**
P.USHA SREE	**22701A3144**
M.SUNIL	**22701A3142**
S.MANOJ KUMAR	**23705A3103**

in partial fulfillment of the requirements for the award of Degree of **Bachelor of Technology** in **"Computer Science and Engineering(AI)"**. Course contains the plagiarism of **18%** which is within the acceptable limits.

Date:

Dean / Coordinator

Research & Development Cell

Date:

DECLARATION

We hereby declare that the project report entitled **ENHANCING EMERGENCY CARE - THROUGH MACHINE LEARNING BASED ADMISSION PRIORITIZATION** under the guidance of **Dr.C.V.Lakshmi Narayana** M.Tech,ph.D., **Assistant Professor, Department of CSE(AI)**, Department of Computer Science and Engineering(AI) is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering(AI).

This is a record of bonafide work carried out by me/us and the results embodied in this project report have not been reproduced or copied from any source. The results embodied in this project report have been submitted to any other University or institute for the Award of any other Degree or Diploma.

PROJECT ASSOCIATES

K.REVATHY(22701A3135)
D.KARTHIK(22701A3122)
P.USHA SREE(22701A3144)
M.SUNIL(22701A3142)
S.MANOJ KUMAR(23705A3103)

ACKNOWLEDGMENT

We endeavor of a long period can be successful only with the advice of many well-wishers. We take this opportunity to express my deep gratitude and appreciation to all those who encouraged me for the successful completion of the project work.

Our heartfelt thanks to our Guide, ***Dr.C.V.LAKSHMI NARAYANA,*** *M.Tech,Ph.D., Assistant Professor in Department of Artificial Intelligence Annamacharya Institute of Technology and Sciences, Rajampet, for his valuable guidance and suggestions in analyzing and testing throughout the period, till the end of the project work completion.*

We wish to express sincere thanks and gratitude to ***Dr.*** **N.Penchalaiah,** *M.Tech,Ph.D., Associate Professor for his encouragement and facilities that were offered to us for carrying out this project.*

We take this opportunity to offer gratefulness to our Principal ***Dr. S.M.V. Narayana****, for providing all sorts of help during the project work.*

We are very much thankful to ***Dr. C. Gangi Reddy****, Honorary Secretary of the Annamacharya Educational Trust, for his help in providing good facilities in our college.*

We would express our sincere thanks to all faculty members of ***Computer Science and Engineering(AI) Department, batch-mates, friends*** *and* ***lab-technicians****, who have helped us to complete the project work successfully.*

Finally, we express our sincere thanks to ***our parents*** *who has provided their heartfelt support and encouragement in the accomplishment to complete this project successfully.*

PROJECT ASSOCIATES

K.REVATHY(22701A3135)
D.KARTHIK(22701A3122)
P.USHA SREE(22701A3144)
M.SUNIL(22701A3142)
S.MANOJ KUMAR(23705A3103)

CONTENTS

CHAPTER	Page No.
ABSTRACT	
1. INTRODUCTION	1-3
1.1. Motivation	2
1.2. Problem Statement	2
1.3. Objective Of the Project	2
1.4. Scope	2
1.5. Project Introduction	2-3
2. LITERATURE SURVEY	4-8
2.1. Related Work	5-8
3. SYSTEM ANALYSIS	9-11
3.1. Existing System	10
3.2. Disadvantages	10
3.3. Proposed System	11
3.4. Advantages	10-11
3.5. Work Flow of Proposed System	11
4. REQUIREMENTS ANALYSIS	12-16
4.1. Feasibility Study	13-14
4.2. Functional and non-functional Requirements	14-15
4.3. Hardware Requirements	15
4.4. Software Requirements	15
4.5. Architecture	16
5. SYSTEM DESIGN	17-28
5.1. Introduction of Input Design	18-19
5.2. UML Diagram	19-28
5.2.1. Use Case Diagram	19-20
5.2.2. Class Diagram	21
5.2.3. Sequence Diagram	21-22
5.2.4. Collaboration Diagram	22-23
5.2.5. Deployment Diagram	23
5.2.6. Activity Diagram	23-24
5.2.7. Component Diagram	24-25
5.2.8. ER Diagram	25
5.3. DFD Diagram	25-28
6. SYSTEM CODING IMPLEMENTATION	29-53

6.1. Module 30-42
6.2. Software Installation 42-46
6.3. Sample Code 46-53
7. SYSTEM STUDY AND TESTING 54-59
7.1. Types of Tests 55-59
7.1.1 Unit Testing 55
7.1.2 Integration Testing 55
7.1.3 Functional Testing 56
7.1.4 White Box Testing 56
7.1.5 Black Box Testing 56-59
8. RESULTS 60-65
8.1. Results 61-65
9. CONCLUSION AND FUTURE ENHANCEMENTS 66-67
10. REFERENCES 68-69
PLAGARISM REPORT(RECEIPT)
CONFERENCE CERTIFICATES
CONFERENCE PAPER
MAPPING WITH PROGRAM OUTCOMES
MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS

LIST OF FIGURES AND TABLES

Fig No	Figures	Page No
3.5	Work flow of Proposed System	11
4.5	Architecture	16
5.2.1	Use Case Diagram	20
5.2.2	Class Diagram	21
5.2.3	Sequence Diagram	22
5.2.4	Collaboration Diagram	23
5.2.5	Deployment Diagram	23
5.2.6	Activity Diagram	24
5.2.7	Component Diagram	25
5.2.8	ER Diagram	25
5.3	DFD Diagram	26
5.3.1	Level 1 Diagram	27
5.3.2	Level 2 Diagram	28
6.1.2	Random Forest	36
8.1	Result	61
8.1.1	Home Page	61
8.1.2	User Registration	61
8.1.3	User Login	62
8.1.4	User Home Page	62
8.1.5	Upload Data	63
8.1.6	View Data	63
8.1.7	Preprocessing	64
8.1.8	Model Page	64
8.1.9	Prediction	65
8.1.10	Check Bed Availability	65

ABSTRACT

The use of artificial intelligence and machine learning techniques in emergency medicine has grown rapidly. This project reviews and assesses studies in this field, categorizing them into three areas: prediction and detection of disease, prediction of need for admission, discharge, and mortality, and machine learning-based triage systems. In the first category, several studies have been conducted using machine learning algorithms. This project used various machine learning techniques, including Logistic Regression, Naive Bias, Random Forest, MLP, SVC, LSTM, and datasets from electronic health records and medical imaging. The second category focuses on predicting the need for hospital admission, discharge, and mortality using machine learning algorithms. This project have used various datasets, including electronic health records, laboratory data, and vital signs. The algorithms used in these studies include decision trees, random forests, and logistic regression models. Finally, the third category explores the development of machine learning-based triage systems for emergency departments. This project have used various datasets, including vital signs and medical history, and machine learning techniques such as decision trees, artificial neural networks, and fuzzy logic. Overall, the studies reviewed in this project demonstrate the potential of artificial intelligence and machine learning techniques in emergency medicine. However, the accuracy and effectiveness of these algorithms depend on the quality and quantity of the data used. Further research is needed to validate the findings and improve the performance of these algorithms in bed setting.

Keywords: *Logistic Regression, Naive Bias, Random Forest, MLP, SVC, LSTM.*

CHAPTER-1

INTRODUCTION

1. INTRODUCTION

1.1 Motivation:

Rapid advancements in AI and machine learning fuel the exploration of their role in emergency medicine. This study investigates their application across three pivotal areas: disease prediction, hospital admission/discharge prognosis, and the development of AI-driven triage systems. The diverse array of techniques and datasets utilized in prior research underscores the potential for enhancing emergency medical practices.

1.2 Problem Statement:

Despite promising strides, the efficacy of AI and ML algorithms in emergency medicine hinges on data quality and quantity. The variance in accuracy and effectiveness across studies reveals the critical need for validation and improvement, especially within clinical settings.

1.3 Objective of the Project:

This review aims to assess the evolving landscape of AI and ML techniques in emergency medicine, delineating their application in disease prediction, hospital admission/discharge prognosis, and the development of triage systems. The primary objective is to highlight the potential while identifying the imperative need for further research and validation.

1.4 Scope:

This project delves into the multifaceted application of AI and ML in emergency medicine, specifically analyzing studies utilizing varied techniques like Logistic Regression, Naive Bias, Random Forest, MLP, SVC, LSTM, and diverse datasets, encompassing electronic health records, medical imaging, vital signs, and medical history. However, its scope underscores the dependency of algorithmic accuracy on data quality, advocating for future research improvements.

1.5 Project Introduction:

Prioritizing hospital admission according to emergency using machine learning is a promising application of artificial intelligence in healthcare. Machine learning algorithms can analyze large amounts of data and identify patterns that can help predict which patients are at higher risk and need immediate attention. There are various factors that can be considered when prioritizing hospital admission for emergency cases. Some of these factors include the patient's age, medical history, symptoms, vital signs, and the severity of their condition. Machine learning algorithms can be trained on historical data to learn which factors are most important in predicting patient outcomes and can be used to create a predictive model that assigns

risk score to each patient. The predictive model can be used to prioritize hospital admission by flagging patients with higher risk scores for immediate attention by medical professionals. This can help ensure that patients receive timely and appropriate care, leading to better outcomes and potentially saving lives. It is important to note that machine learning algorithms are not a substitute for medical professionals' expertise and judgment. The model should be used as an aid in decision-making and not as the sole basis for admitting patients to the hospital. Additionally, ethical considerations should be taken into account when using predictive models in healthcare, such as ensuring fairness and avoiding bias in algorithmic decision-making. Overall, machine learning has the potential to improve emergency care by enabling more efficient and accurate triage of patients in need of immediate attention.

CHAPTER-2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Related Work:

[1] H. Xu, W. Wu, S. Nemati, and H. Zha,

"Patient Flow Prediction via Discriminative Learning of Mutually-Correcting Processes," IEEE Trans. on Knowl. and Data Engineering,vol. 29, no. 1, pp. 157–171, Jan. 2017.

As occupational rehabilitation services are part of the public medical and health services in Hong Kong, work-injured workers are treated along with other patients and are not considered a high priority for occupational rehabilitation services. The idea of a work trial arrangement in the private market occurred to meet the need for a more coordinated occupational rehabilitation practice. However, there is no clear service standard in private occupational rehabilitation services nor concrete suggestions on how to offer rehabilitation plans to injured workers. Electronic Health Records (EHRs) data can provide a foundation for developing a model to improve this situation. This project aims at using a machine-learning-based approach to enhance the traditional prediction of disability duration and rehabilitation plans for work-related injury and illness. To help patients and therapists to understand the machine learning result, we also developed an interactive dashboard to visualize machine learning results. The outcome is promising. Using the variational autoencoder, our system performed better in predicting disability duration. We have around 30% improvement compared with the human prediction error. We also proposed further development to construct a better system to manage the work injury case.

Summary: This project introduced a smart work injury system. To explain the wounded case path in a high-dimensional environment, we developed the rose garden conceptual model. According to the conceptual model, a NN model was trained to project the incoming case into the high-dimensional latent space.

2. P. R. E. Jarvis, "Improving emergency department patient flow," Clin. Exp Emergency Med., vol. 3, no. 2, p. 63, 2016.

Emergency departments (ED) face significant challenges in delivering high quality and timely patient care on an ever-present background of increasing patient numbers and limited hospital resources. A mismatch between patient demand and the ED's capacity to deliver care often leads to poor patient flow and departmental crowding. These are associated with reduction in the quality of the care delivered and poor patient outcomes. A literature review was performed to identify evidence-based strategies to reduce the amount of time patients spend in the ED in order to improve patient flow and reduce crowding in the ED. The use of doctor triage, rapid

assessment, streaming and the co-location of a primary care clinician in the ED have all been shown to improve patient flow. In addition, when used effectively point of care testing has been shown to reduce patient time in the ED. Patient flow and departmental crowding can be improved by implementing new patterns of working and introducing new technologies such as point of care testing in the ED.

Summary: In this project Crowding brought on by poor patient flow severely limits the ED's ability to provide high-quality emergency and urgent care. Patient flow in the ED is primarily impacted by long wait times for patients, sluggish investigation turnaround times, and delays in disposition decisions.

[3] A. Staib, C. Sullivan, J. B. Prins, A. Burton-Jones, G. Fitzgerald, and I.Scott, "Uniting emergency and inpatient clinicians across the ed–inpatient interface: The last frontier?" Emergency Med. Australasia, vol. 29, no. 6, pp. 740–745, 2017.

The purpose of this project is to demonstrate through practical application how a system dynamics (SD) patient flow model of an emergency department (ED) can show which levers effectively reduce backlogs to improve access to care. Overcrowded EDs are struggling to meet demand and access targets. In 2016 and 2017, in the UK and Australia, respectively, 15% and 28% of arrivals waited longer than the targeted 4-hr treatment time. Historically, simulation models that have informed access to emergency care have ignored the wider systems impacts. There is a growing awareness of the value of systems analysis tools for informing interventions and policy. In this study, we constructed a pilot system dynamics patient flow model, where the scope was the ambulatory and ambulance patient arrivals, the ED processes for acute and fast-track pathways, pathology and radiology services, the ED short-stay unit, and the Medical Assessment Planning Inpatient Unit. Patients queued to access constrained ED resources (doctors and beds) and diagnostic services (pathology and X-ray). The model was tested on actual data from five separate historical periods spanning 3 years. The resultant daily pattern of peaks and troughs in patient flow and system delays accurately replicated patterns in actual patient flows, resource use, and the location of delays. "What if" scenario analysis (b) simulated how access would have looked in the sample weeks with different intervention strategies, (b) simulated system limits on the basis of current resources, (c) accurately identified levers that historically have been most effective at minimizing ambulance ramping, and (d) identified when additional staffing would fail to improve flow.

Summary: This study aims to illustrate through application how a system dynamics (SD) patient flow model of an emergency department (ED) can identify which levers efficiently minimize backlogs to enhance access to care.

[4] O. Karan, C. Bayraktar, H. Gumu¸skaya, and B. Karlik, "Diagnosing diabetes using neural networks on small mobile devices," Expert Syst. Appl., vol. 39, no. 1, pp. 54–60, 2012

Pervasive computing is often mentioned in the context of improving healthcare. This project presents a novel approach for diagnosing diabetes using neural networks and pervasive healthcare computing technologies. The recent developments in small mobile devices and wireless communications provide a strong motivation to develop new software techniques and mobile services for pervasive healthcare computing. A distributed end-to-end pervasive healthcare system utilizing neural network computations for diagnosing illnesses was developed. This work presents the initial results for a simple client (patient's PDA) and server (powerful desktop PC) two-tier pervasive healthcare architecture. The computations of neural network operations on both client and server sides and wireless network communications between them are optimized for real time use of pervasive healthcare services.

Summary: This study Pervasive computing is frequently discussed in relation to enhancing healthcare. In this project, a novel method for detecting diabetes using pervasive healthcare computing and neural networks is presented.

[5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in Proc. 26th Annu. Int. Conf. Mach. Learn, 2009, pp. 41–48.

Humans and animals learn much better when the examples are not randomly presented but organized in a meaningful order which illustrates gradually more concepts, and gradually more complex ones. Here, we formalize such training strategies in the context of machine learning, and call them "curriculum learning". In the context of recent research studying the difficulty of training in the presence of non-convex training criteria (for deep deterministic and stochastic neural networks), we explore curriculum learning in various set-ups. The experiments show that significant improvements in generalization can be achieved. We hypothesize that curriculum learning has both an effect on the speed of convergence of the training process to a minimum and, in the case of non-convex criteria, on the quality of the local minima obtained: curriculum learning can be seen as a particular form of continuation method (a general strategy for global optimization of non-convex functions).

Summary: In this project When examples are structured in a purposeful order that gradually demonstrates additional concepts rather than being offered at random, both humans and animals learn considerably more effectively..

[6] V. A. Convertino et al., "Use of advanced machine-learning techniques for noninvasive monitoring of hemorrhage," J. Trauma Acute Care Surgery, Vol. 71, no. 1, pp. S25–S32, 2011.

Uncontrolled post-traumatic hemorrhage is an important cause of traumatic mortality that can be avoided. This study intends to use machine learning (ML) to build an algorithm based on data collected from an electronic health record (EHR) system to predict the risk of delayed bleeding in trauma patients in the ICU. We enrolled patients with torso trauma in the surgical ICU. Demographic features, clinical presentations, and laboratory data were collected from EHR. The algorithm was designed to predict hemoglobin dropping 6 h before it happened and evaluated the performance with 10-fold cross-validation. We collected 2218 cases from 2008 to 2018 in a trauma center. There were 1036 (46.7%) patients with positive hemorrhage events during their ICU stay. Two machine learning algorithms were used to predict ongoing hemorrhage events. The logistic model tree (LMT) and the random forest algorithm achieved an area under the curve (AUC) of 0.816 and 0.809, respectively. In this study, we presented the ML model using demographics, vital signs, and lab data, promising results in predicting delayed bleeding risk in torso trauma patients. Our study also showed the possibility of an early warning system alerting ICU staff that trauma patients need re-evaluation or further survey.

Summary: In this study, an key factor in traumatic mortality that can be prevented is uncontrolled post-traumatic bleeding. The goal of this study is to develop an algorithm based on data from an electronic health record (EHR) system to predict the likelihood of delayed bleeding in trauma patients in the intensive care unit (ICU).

.

CHAPTER-3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Existing System

In the existing system, implementation of machine learning algorithms is bit complex to build due to the lack of information about the data visualization. Mathematical calculations are used in existing system for Logistic Regression, Random Forest, SVC model building this may takes the lot of time and complexity. To overcome all this, we use machine learning packages available in the scikit-learn library

3.2 Disadvantages:

i. **Dependency on Data Quality**: Performance heavily relies on the quality and completeness of input data, impacting model accuracy.
ii. **Black Box Nature**: Interpretability can be challenging, as some models lack transparency in explaining predictions.
iii. **Limited Feature Engineering**: Automated processes may overlook nuanced feature engineering crucial for specific datasets.
iv. **Algorithm Selection Bias**: Relying solely on library defaults might lead to suboptimal algorithm choices for specific tasks.
v. **Overfitting Risk:** Automatic hyperparameter tuning may increase the risk of overfitting, requiring additional validation and fine-tuning.

3.3 Proposed System

Proposed several machine learning models to classify the hospital admission according to emergency, but none have adequately addressed this misdiagnosis problem. Also, similar studies that have proposed models for evaluation of such Hospital Emergency case and the size of the data Therefore, we propose Naive Bias, MLP, and LSTM performing classifier tests based.

3.4 Advantages:

i. **Robust Adaptability**: Naive Bayes exhibits robustness to irrelevant features, while MLP adapts to diverse data distributions, and LSTM models temporal dependencies effectively.
ii. **Efficiency Spectrum:** Naive Bayes offers computational efficiency, MLP handles complex patterns efficiently, and LSTM retains long-term information while handling sequential data adeptly.

iii. **Data Versatility**: Naive Bayes performs well with smaller datasets, MLP scales effectively with larger datasets, and LSTM grasps contextual understanding within sequences.

iv. **Automatic Learning and Retention**: All three models showcase an ability for automatic feature learning—Naive Bayes with redundant information, MLP with automatic representation learning, and LSTM with automatic feature extraction and retention.

v. **Domain Adaptation**: Naive Bayes maintains performance across different data types, MLP exhibits versatility across various problem domains, and LSTM effectively models dependencies, especially in sequential data, ensuring adaptability across different contexts.

3.5 work Flow of Proposed system

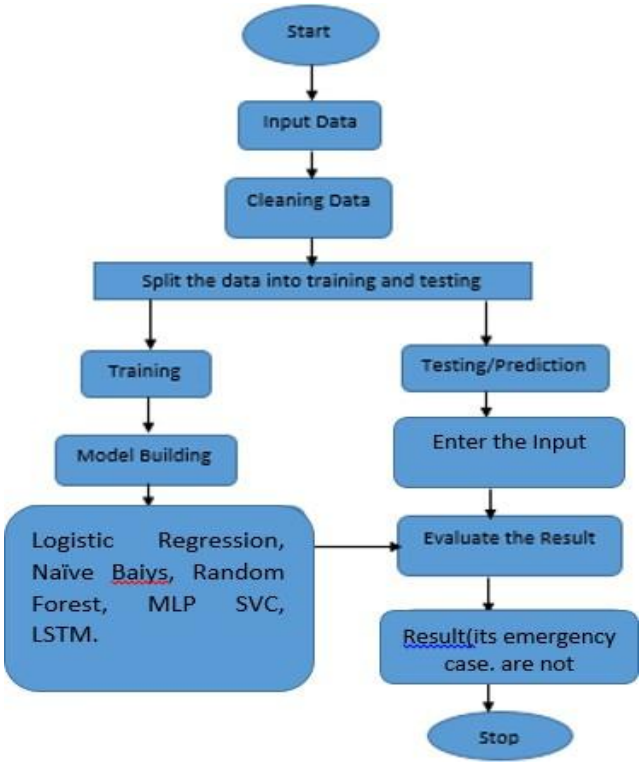


Fig 3.5 : Work Flow of Proposed System.

CHAPTER-4

REQUIREMENT ANALYSIS

4. REQUIREMENTS ANALYSIS

4.1 Feasibility Study

The feasibility of the project is analysed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- Economical feasibility
- Technical feasibility
- Social feasibility

Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and

to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

System Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

4.2 Functional and non-functional requirements

Requirement's analysis is very critical process that enables the success of a system or software project to be assessed. Requirements are generally split into two types: Functional and non-functional requirements.

Functional Requirements: These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

Examples of functional requirements:

i. Authentication of user whenever he/she logs into the system
ii. System shutdown in case of a cyber-attack
iii. A verification email is sent to user whenever he/she register for the first time on some software system.

Non-functional requirements: These are basically the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements.

They basically deal with issues like:

- Portability
- Security
- Maintainability
- Reliability
- Scalability
- Performance
- Reusability
- Flexibility

Examples of non-functional requirements:

i. Emails should be sent with a latency of no greater than 12 hours from such an activity.
ii. The processing of each request should be done within 10 seconds
iii. The site should load in 3 seconds whenever of simultaneous users are > 10000

4.3 Hardware Requirements:

Processor	**- I7/Intel Processor**
Hard Disk	- 160GB
Key Board	- Standard Windows Keyboard
Mouse	- Two or Three Button Mouse
Monitor	- SVGA
RAM	- 8GB

4.4 Software Requirements:

Operating System	: Windows 11
Server side Script	: HTML, CSS & JS
Programming Language	: Python
Libraries	: Django, Pandas, MySQL, OS, Smtplib, Numpy
IDE/Workbench	: PyCharm
Technology	: Python 3.6+

4.5 Architecture:

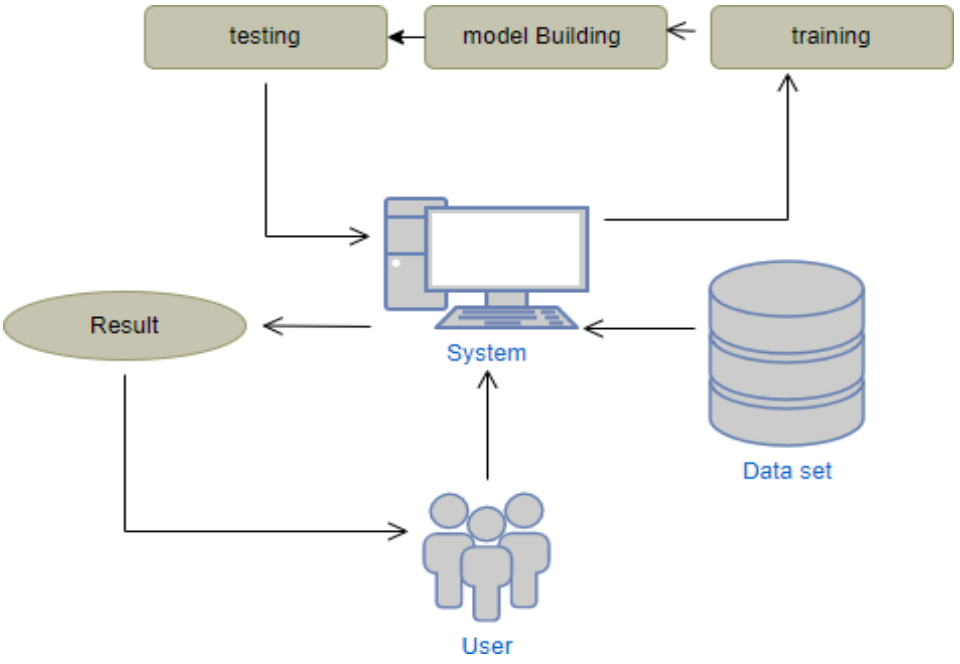


Fig 4.5: Architecture.

CHAPTER-5

SYSTEM DESIGN

5. SYSTEM DESIGN

5.1 Introduction of Input Design:

In an information system, input is the raw data that is processed to produce output. During the input design, the developers must consider the input devices such as PC, MICR, OMR, etc.

Therefore, the quality of system input determines the quality of system output. Well-designed input forms and screens have following properties −

- It should serve specific purpose effectively such as storing, recording, and retrieving the information.
- It ensures proper completion with accuracy.
- It should be easy to fill and straightforward.
- It should focus on user's attention, consistency, and simplicity.
- All these objectives are obtained using the knowledge of basic design principles regarding −
    - What are the inputs needed for the system?
    - How end users respond to different elements of forms and screens.

Objectives for Input Design:

The objectives of input design are −

- To design data entry and input procedures
- To reduce input volume
- To design source documents for data capture or devise other data capture methods
- To design input data records, data entry screens, user interface screens, etc.
- To use validation checks and develop effective input controls.

Output Design:

The design of output is the most important task of any system. During output design, developers identify the type of outputs needed, and consider the necessary output controls and prototype report layouts.

Objectives of Output Design:

The objectives of input design are:

- To develop output design that serves the intended purpose and eliminates the production of unwanted output.
- To develop the output design that meets the end user's requirements.
- To deliver the appropriate quantity of output.
- To form the output in appropriate format and direct it to the right person.
- To make the output available on time for making good decisions.

5.2 UML Diagrams:

5.2.1 Use Case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

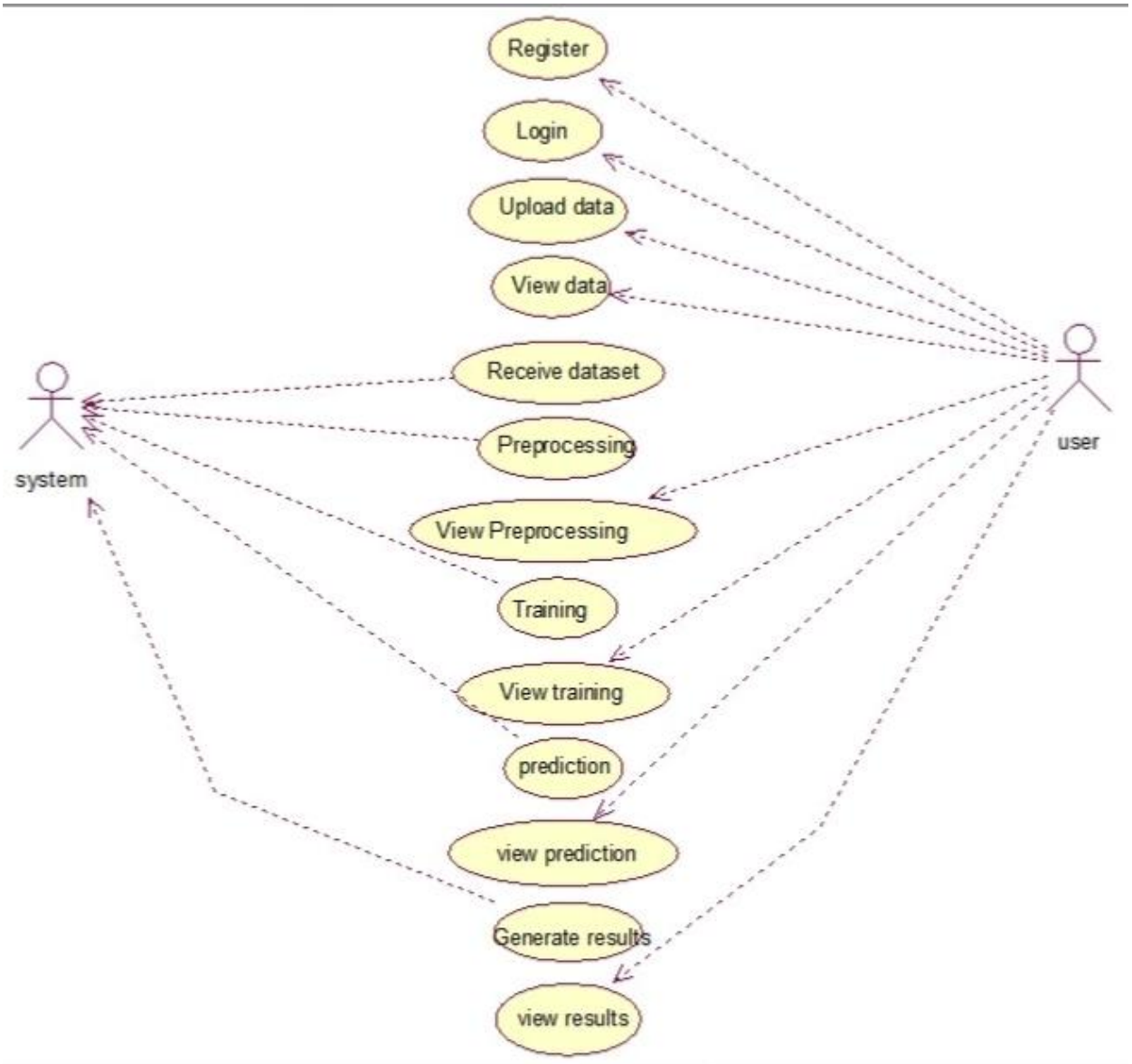


Fig 5.2.1: Use Case Diagram

5.2.2 Class Diagram:

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

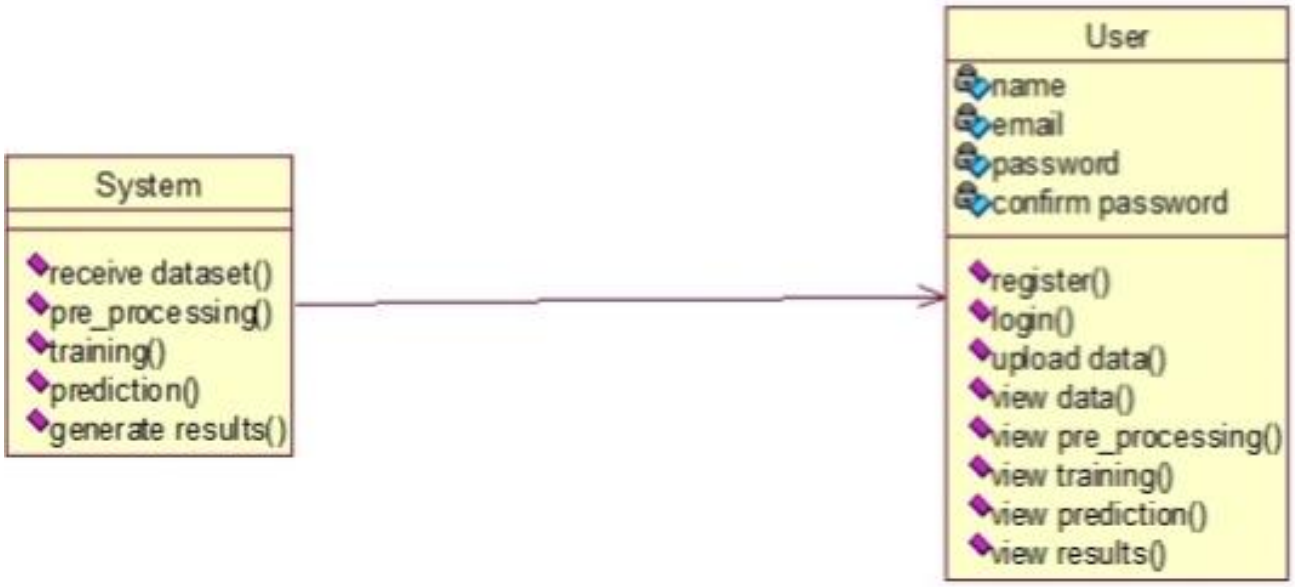


Fig 5.2.2: Class Diagram

5.2.3 Sequence Diagram:

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

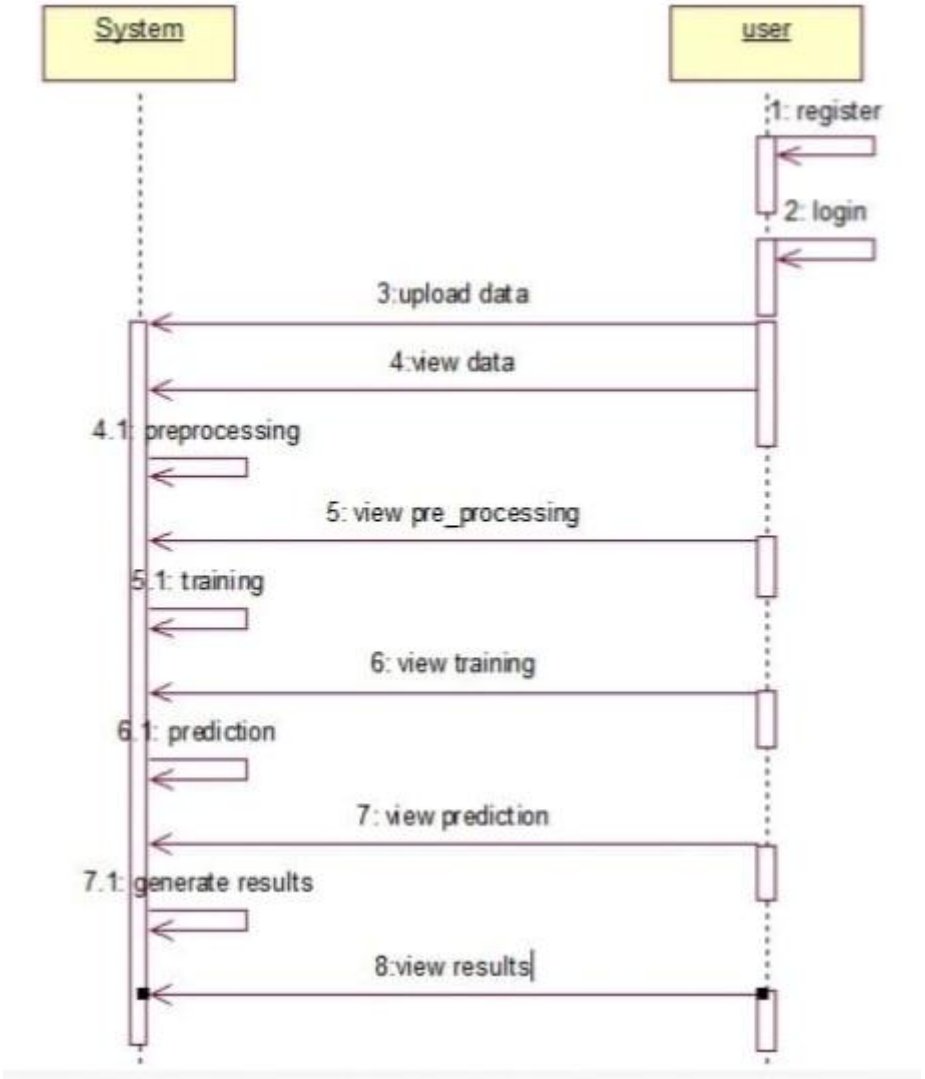


Fig 5.2.3: Sequence Diagram

5.2.4 Collaboration Diagram:

In collaboration diagram the method call sequence is indicated by some numbering technique as shown below. The number indicates how the methods are called one after another. We have taken the same order management system to describe the collaboration diagram. The method calls are similar to that of a sequence diagram. But the difference is that the sequence diagram does not describe the object organization whereas the collaboration diagram shows the object organization.

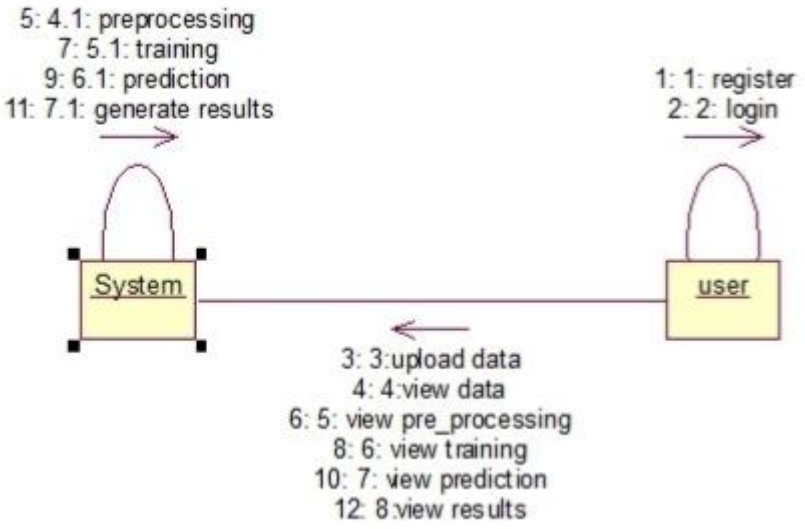


Fig 5.2.4: Collaboration Diagram

5.2.5 Deployment Diagram

Deployment diagram represents the deployment view of a system. It is related to the component diagram. Because the components are deployed using the deployment diagrams. A deployment diagram consists of nodes. Nodes are nothing but physical hardware's used to deploy the application.



Fig 5.2.5: Deployment Diagram

5.2.6 Activity Diagram:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modelling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

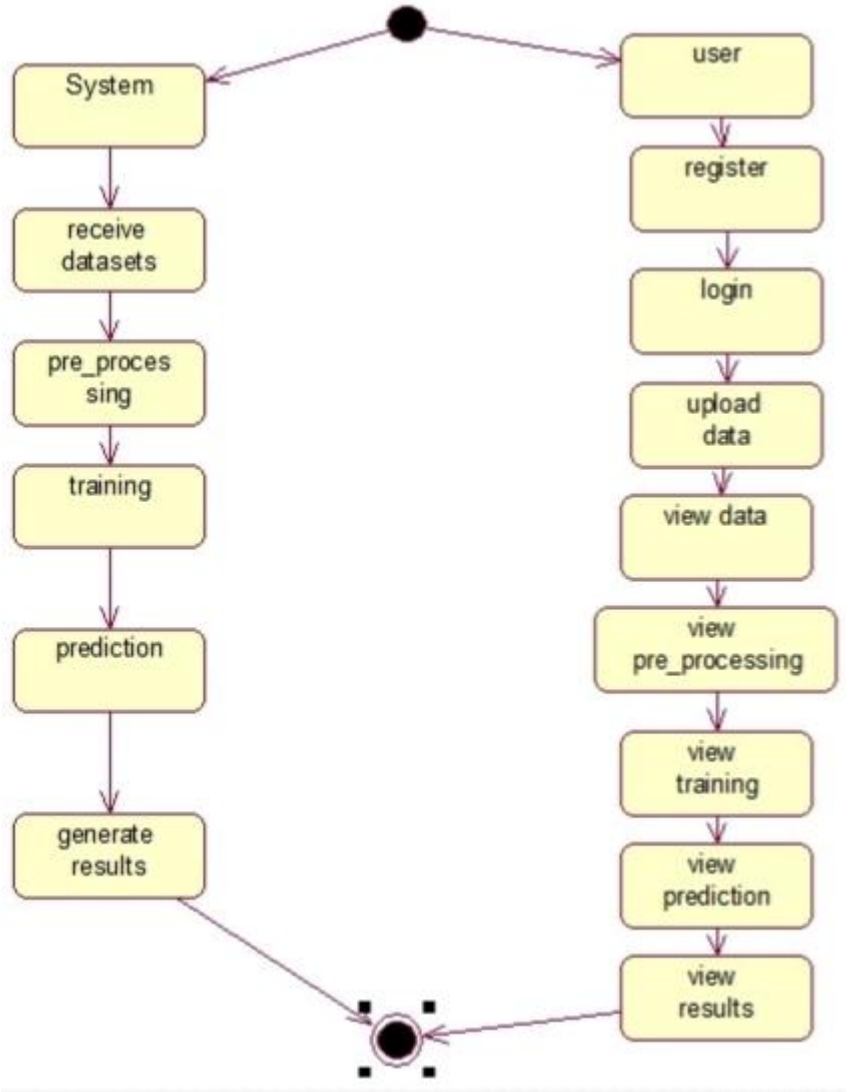


Fig 5.2.6: Activity Diagram

5.2.7 Component Diagram:

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical **c**omponents in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.



Fig 5.2.7: Component Diagram

5.2.8 ER Diagram:

An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram). An ER model is a design or blueprint of a database that can later be implemented as a database. The main components of E-R model are: entity set and relationship set.

An ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables and their attributes, ER diagram shows the complete logical structure of a database. Let's have a look at a simple ER diagram to understand this concept.

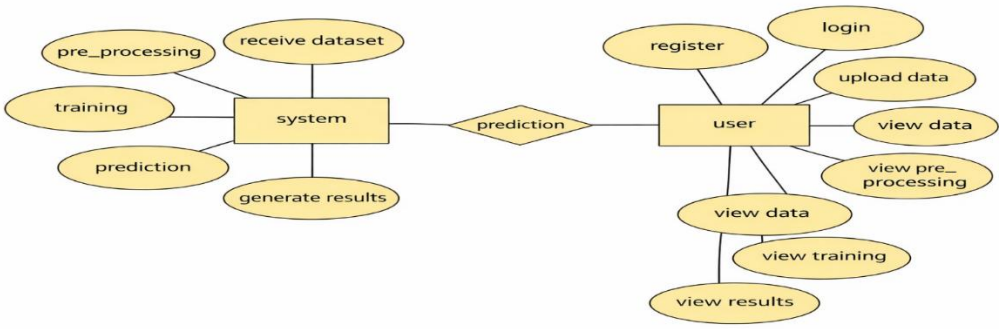


Fig 5.2.8: ER Diagram

5.3 DFD Diagram:

A Data Flow Diagram (DFD) is a traditional way to visualize the information flows within a system. A neat and clear DFD can depict a good amount of the system requirements graphically. It can be manual, automated, or a combination of both. It shows how information enters and leaves the system, what changes the information and where information is stored. The purpose of a DFD is to show the scope and boundaries of a system as a whole. It may be used a communications tool between a systems analyst and any person who plays a part in the system

that acts as the starting point for redesigning a system.

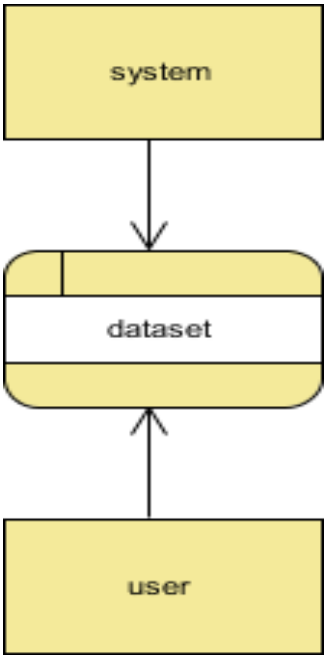


Fig 5.3: DFD Diagram

5.3.1 Level 1 Diagram:

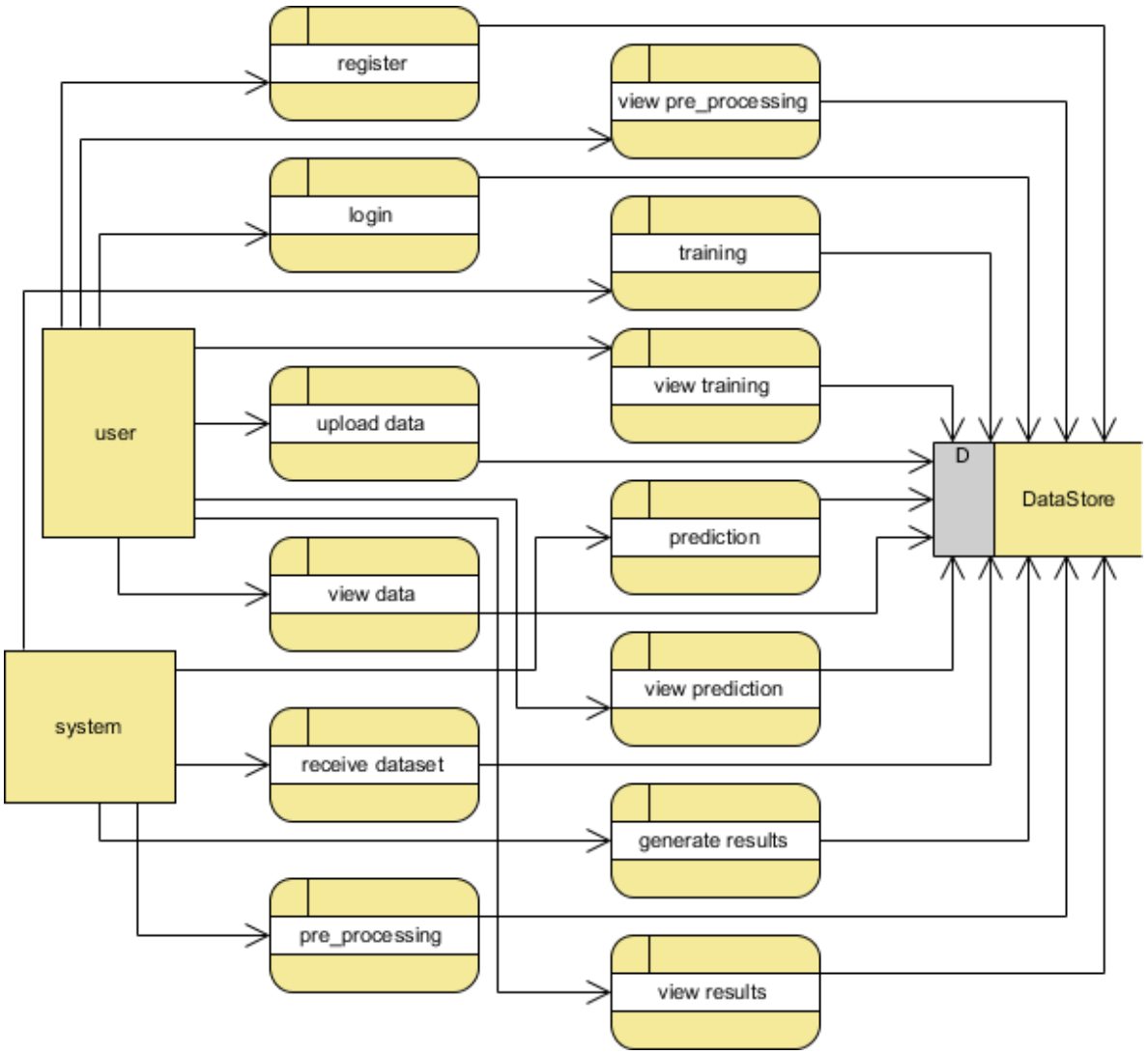


Fig 5.3.1: Level 1 Diagram

5.3.2 Level 2 Diagram:

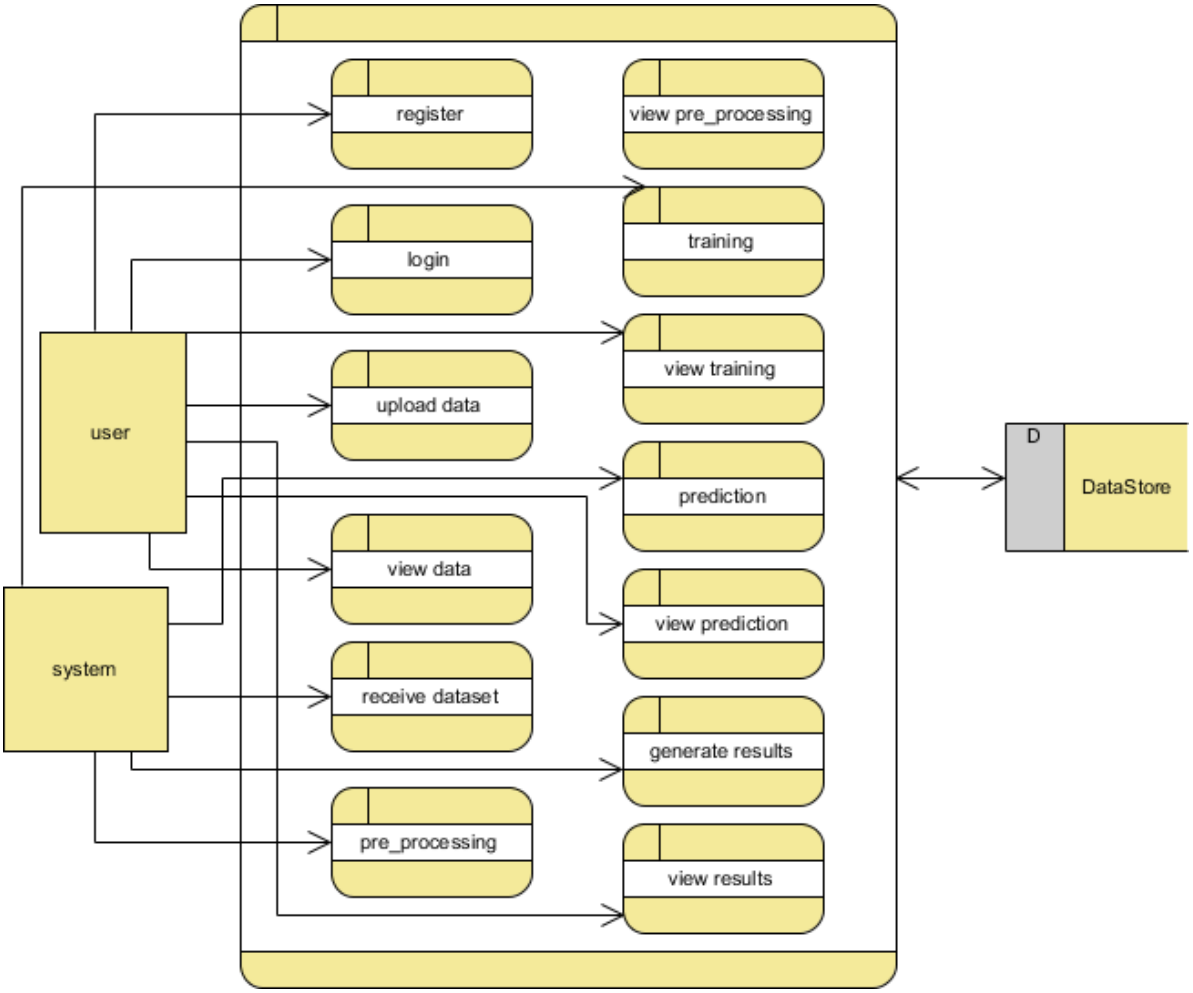


Fig 5.3.2: Level 2 Diagram

CHAPTER 6

SYSTEM CODING AND IMPLEMENTATION

6. SYSTEM CODING AND IMPLEMENTATION

6.1 Modules:

6.1.1 **User**:

6.1.2 **Register:**

Users can register for the Hospital web application here.

6.1.3 **Login:**

After registering, the user can access his portal.

6.1.4 **Upload:**

Upload dataset which is downloaded from the Kaggle

6.1.5 **View Data:**

View data before preprocessing

6.1.6 **Preprocessing Data:**

User Can View After Preprocessing Data

6.1.7 **Input :**

User will give the input values.

6.1.8 **Result History:**

After giving the inputs, model will predict the result which it was set according to performance, it will predict that the Hospital Emergency is it's a emergency case are its not emergency case.

Take Dataset:

The dataset for the ED Admission Dataset is collected from the kaggle website (kaggle.com). The size of overall dataset is 9.08 MB.

Pre-processing:

- In preprocessing first of all we will check whether there is any Nan values.
- If any Nan values is present we will fill the Nan values with different fillna techniques like bfill, ffill, mode, and mean.
- Here we used the ffill (front fill) technique on our project.

Training the data:

i. Irrespective of the algorithm we select the training is the same for every algorithm**.**

ii. Given a dataset we split the data into two parts training and testing, the reason behind doing this is to test our model/algorithm performance just like the exams for a student the testing is also exam for the model.

iii. We can split data into anything we want but it is just good practice to split the data such

that the training has more data than the testing data, we generally split the data.

iv. And for training and testing there are two variables X and Y in each of them, the X is the features that we use to predict the Y target and same for the testing also.

v. Then we call the .fit ( ) method on any given algorithm which takes two parameters i.e., X and Y for calculating the math and after that when we call the .predict ( ) giving our testing X as parameter and checking it with the accuracy score giving the testing Y and predicted X as the two parameters will get our accuracy score and same steps , these are just checking for how good our model performed on a given dataset..

METHODOLOGY AND ALGORITHMS:

1. SUPPORT VECTOR CLASSIFIER(SVC):

The objective of the support vector machine algorithm is to find a hyper plane in an N-dimensional space (N — the number of features) that distinctly classifies the data points.

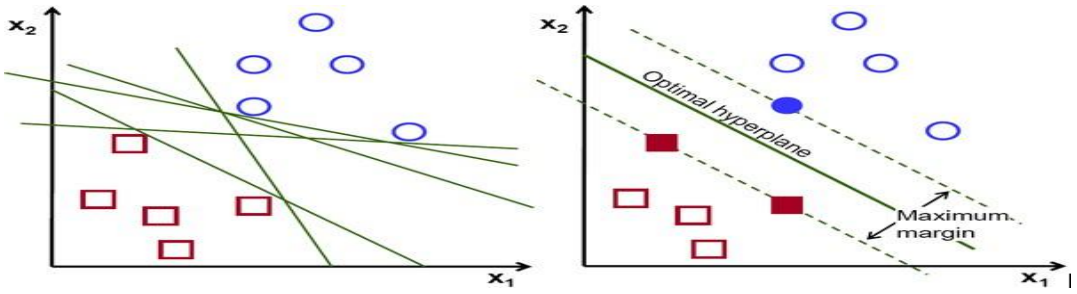


Fig 6.1.1.1: Support Vector Classifier

Possible hyper planes :

To separate the two classes of data points, there are many possible Hyper planes that could be chosen. Our objective is to find a plane that has the maximum margin, i.e. the maximum distance between data points of both classes. Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence.

Hyper planes and Support Vectors

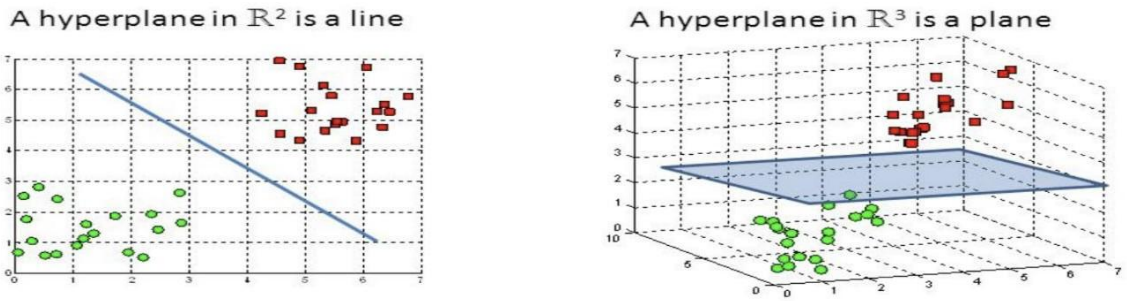


Fig 6.1.1.2: Hyper planes and Support Vectors

Hyper planes in 2D and 3D feature space

Hyper planes are decision boundaries that help classify the data points. Data points falling on either side of the hyper plane can be attributed to different classes. Also, the dimension of the hyper plane depends upon the number of features. If the number of input features is 2, then the hyper plane is just a line. If the number of input features is 3, then the hyper plane becomes a two-dimensional plane. It becomes difficult to imagine when the number of features exceeds 3.

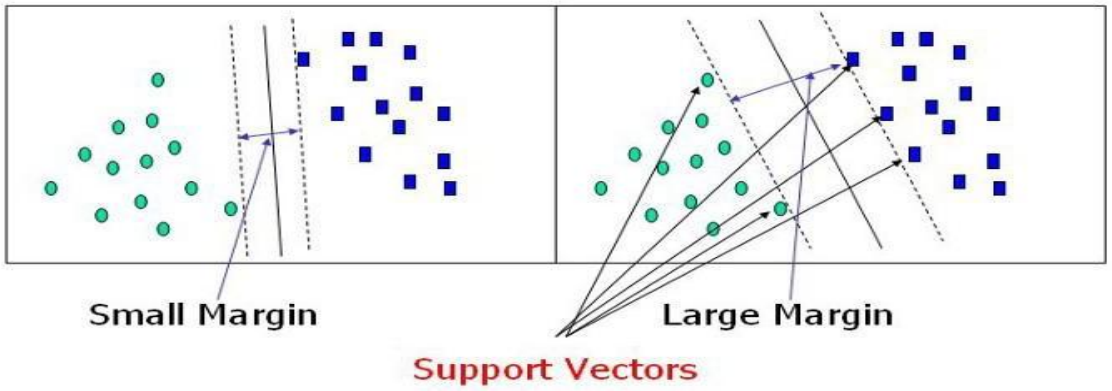


Fig 6.1.1.3(a): Hyper planes in 2D and 3D feature space

Support Vectors

Support vectors are data points that are closer to the hyper plane and influence the position and orientation of the hyper plane. Using these support vectors, we maximize the margin of the classifier. Deleting the support vectors will change the position of the hyper plane. These are the points that help us build our SVM.

Large Margin Intuition

In logistic regression, we take the output of the linear function and squash the value within the range of [0,1] using the sigmoid function. If the squashed value is greater than a threshold value (0.5) we assign it a label 1, else we assign it a label 0. In SVM, we take the output of the linear function and if that output is greater than 1, we identify it with one class and if the output is -1, we identify is with another class. Since the threshold values are changed to 1 and -1 in SVM, we obtain this reinforcement range of values ([-1, 1]) which acts as margin.

Cost Function and Gradient Updates

In the SVM algorithm, we are looking to maximize the margin between the data points and the hyper plane. The loss function that helps maximize the margin is hinge loss.

$$c(x, y, f(x)) = \begin{cases} 0, & \text{if } y * f(x) \geq 1 \\ 1 - y * f(x), & \text{else} \end{cases}$$

$$c(x, y, f(x)) = (1 - y * f(x))_+$$

Hinge loss function (function on left can be represented as a function on the right)
The cost is 0 if the predicted value and the actual value are of the same sign. If they are not, we then calculate the loss value. We also add a regularization parameter the cost function.

$$min_w \lambda \parallel w \parallel^2 + \sum_{i=1}^{n} (1 - y_i \langle x_i, w \rangle)_+$$

The objective of the regularization parameter is to balance the margin maximization and loss. After adding the regularization parameter, the cost functions looks as below.

Loss function for SVM

Now that we have the loss function, we take partial derivatives with respect to the weights to find the gradients. Using the gradients, we can update our weights.

$$\frac{\delta}{\delta w_k} \lambda \parallel w \parallel^2 = 2\lambda w_k$$

$$\frac{\delta}{\delta w_k} \left(1 - y_i \langle x_i, w \rangle\right)_+ = \begin{cases} 0, & \text{if } y_i \langle x_i, w \rangle \geq 1 \\ -y_i x_{ik}, & \text{else} \end{cases}$$

When there is no misclassification, i.e. our model correctly predicts the class of our data point, we only have to update the gradient from the regularization parameter.

$$w = w - \alpha \cdot (2\lambda w)$$

Gradient Update — No misclassification
When there is a misclassification, i.e. our model make a mistake on the prediction of the class of our data point, we include the loss along with the regularization parameter to perform gradient update.

$$w = w + \alpha \cdot (y_i \cdot x_i - 2\lambda w)$$

2. Random Forest :

A random forest is a machine learning technique that's used to solve regression and classification problems. It utilizes ensemble learning, which is a technique that combines many classifiers to provide solutions to complex problems.

A random forest algorithm consists of many decision trees. The 'forest' generated by the random forest algorithm is trained through bagging or bootstrap aggregating. Bagging is an ensemble meta-algorithm that improves the accuracy of machine learning algorithms.

The (random forest) algorithm establishes the outcome based on the predictions of the decision trees. It predicts by taking the average or mean of the output from various trees. Increasing the number of trees increases the precision of the outcome.

A random forest eradicates the limitations of a decision tree algorithm. It reduces the over fitting of datasets and increases precision. It generates predictions without requiring many configurations in packages (like Scikit-learn).

Features of a Random Forest Algorithm:

- It's more accurate than the decision tree algorithm.
- It provides an effective way of handling missing data.
- It can produce a reasonable prediction without hyper-parameter tuning.
- It solves the issue of over fitting in decision trees.
- In every random forest tree, a subset of features is selected randomly at the node's splitting point.

Decision trees are the building blocks of a random forest algorithm. A decision tree is a decision support technique that forms a tree-like structure. An overview of decision trees will help us understand how random forest algorithms work.

A decision tree consists of three components: decision nodes, leaf nodes, and a root node. A decision tree algorithm divides a training dataset into branches, which further segregate into other branches. This sequence continues until a leaf node is attained. The leaf node cannot be segregated further.

The nodes in the decision tree represent attributes that are used for predicting the outcome. Decision nodes provide a link to the leaves. The following diagram shows the three types of nodes in a decision tree.

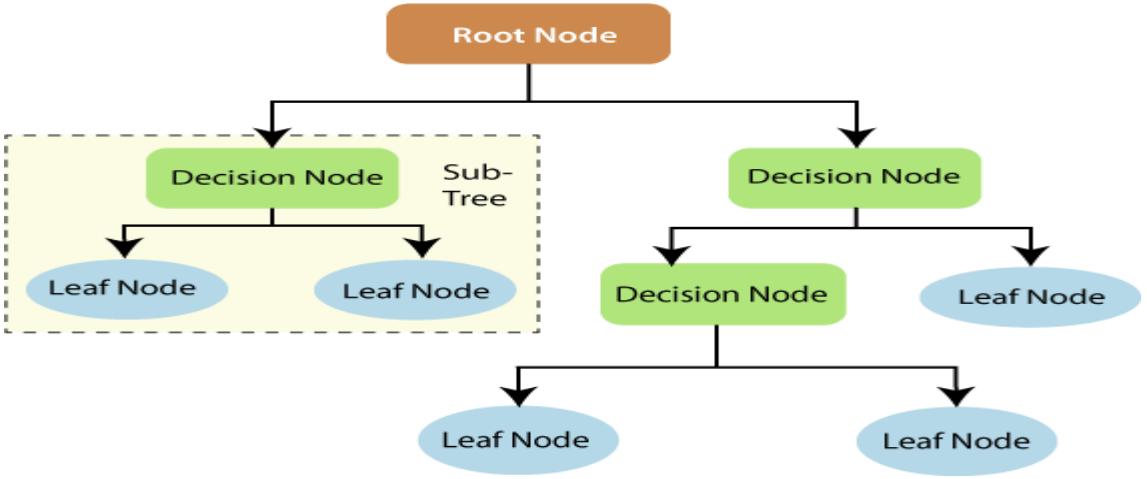


Fig 6.1.1.3(b): Hyper planes in 2D and 3D feature space

The information theory can provide more information on how decision trees work. Entropy and information gain are the building blocks of decision trees. An overview of these fundamental concepts will improve our understanding of how decision trees are built.

Entropy is a metric for calculating uncertainty. Information gain is a measure of how uncertainty in the target variable is reduced, given a set of independent variables.

The information gain concept involves using independent variables (features) to gain information about a target variable (class). The entropy of the target variable (Y) and the conditional entropy of Y (given X) are used to estimate the information gain. In this case, the conditional entropy is subtracted from the entropy of Y.

Information gain is used in the training of decision trees. It helps in reducing uncertainty in these trees. A high information gain means that a high degree of uncertainty (information entropy) has been removed. Entropy and information gain are important in splitting branches, which is an important activity in the construction of decision trees.

Let's take a simple example of how a decision tree works. Suppose we want to predict if a customer will purchase a mobile phone or not. The features of the phone form the basis of his decision. This analysis can be presented in a decision tree diagram.

The root node and decision nodes of the decision represent the features of the phone mentioned above. The leaf node represents the final output, either *buying* or *not buying*. The main features that determine the choice include the price, internal storage, and Random Access Memory

(RAM). The decision tree will appear as follows.

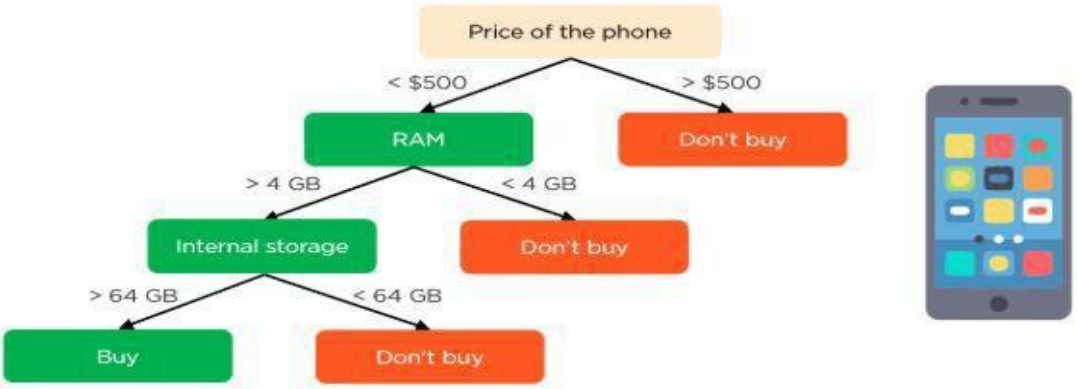


Fig 6.1.2: Random Forest.

The main difference between the decision tree algorithm and the random forest algorithm is that establishing root nodes and segregating nodes is done randomly in the latter. The random forest employs the bagging method to generate the required prediction.

Bagging involves using different samples of data (training data) rather than just one sample. A training dataset comprises observations and features that are used for making predictions. The decision trees produce different outputs, depending on the training data fed to the random forest algorithm. These outputs will be ranked, and the highest will be selected as the final output.

Our first example can still be used to explain how random forests work. Instead of having a single decision tree, the random forest will have many decision trees. Let's assume we have only four decision trees. In this case, the training data comprising the phone's observations and features will be divided into four root nodes.

The root nodes could represent four features that could influence the customer's choice (price, internal storage, camera, and RAM). The random forest will split the nodes by selecting features randomly. The final prediction will be selected based on the outcome of the four trees.

The outcome chosen by most decision trees will be the final choice. If three trees predict *buying*, and one tree predicts *not buying*, then the final prediction will be *buying*. In this case, it's predicted that the customer will buy the phone.

3. LOGISTIC REGRESSION

Logistic Regression was used in the biological sciences in early twentieth century. It was then used in many social science applications. Logistic Regression is used when the dependent variable (target) is categorical.

For example,

To predict whether an email is spam (1) or (0)Whether the tumor is malignant (1) or not (0) Consider a scenario where we need to classify whether an email is spam or not. If we use linear regression for this problem, there is a need for setting up a threshold based on which classification can be done. Say if the actual class is malignant, predicted continuous value 0.4 and the threshold value is 0.5, the data point will be classified as not malignant which can lead to serious consequence in real time.

From this example, it can be inferred that linear regression is not suitable for classification problem. Linear regression is unbounded, and this brings logistic regression into picture. Their value strictly ranges from 0 to 1.

Purpose and examples of logistic regression:

Logistic regression is one of the most commonly used machine learning algorithms for binary classification problems, which are problems with two class values, including predictions such as “this or that,” “yes or no” and “A or B.”

The purpose of logistic regression is to estimate the probabilities of events, including determining a relationship between features and the probabilities of particular outcomes.

One example of this is predicting if a student will pass or fail an exam when the number of hours spent studying is provided as a feature and the variables for the response has two values: pass and fail.

Organizations can use insights from logistic regression outputs to enhance their business strategies so they can achieve their business goals, including reducing expenses or losses and increasing ROI in marketing campaigns, for example.

An e-commerce company that mails expensive promotional offers to customers would like to know whether a particular customer is likely to respond to the offers or not. For example, they’ll want to know whether that consumer will be a “responder” or a “non responder.” In marketing, this is called propensity to respond modeling.

Likewise, a credit card company develops a model to decide whether to issue a credit card to a customer or not will try to predict whether the customer is going to default or not on the credit card based on such characteristics as annual income, monthly credit card payments and number of defaults. In banking parlance, this is known as default propensity modeling.

Uses of logistic regression:

Logistic regression has become particularly popular in online advertising, enabling

marketers to predict the likelihood of specific website users who will click on particular advertisements as a yes or no percentage.

- Logistic regression can also be used in:
- Healthcare to identify risk factors for diseases and plan preventive measures.
- Weather forecasting apps to predict snowfall and weather conditions.
- Voting apps to determine if voters will vote for a particular candidate.

Insurance to predict the chances that a policy holder will die before the term of the policy expires based on certain criteria, such as gender, age and physical examination.
Banking to predict the chances that a loan applicant will default on a loan or not, based on annual income, past defaults and past debts.

Logistic regression vs. linear regression:

The main difference between logistic regression and linear regression is that logistic regression provides a constant output, while linear regression provides a continuous output.
In logistic regression, the outcome, such as a dependent variable, only has a limited number of possible values. However, in linear regression, the outcome is continuous, which means that it can have any one of an infinite number of possible values.

Logistic regression is used when the response variable is categorical, such as yes/no, true/false and pass/fail. Linear regression is used when the response variable is continuous, such as number of hours, height and weight.

For example, given data on the time a student spent studying and that student's exam scores, logistic regression and linear regression can predict different things. With logistic regression predictions, only specific values or categories are allowed. Therefore, logistic regression can predict whether the student passed or failed. Since linear regression predictions are continuous, such as numbers in a range, it can predict the student's test score on a scale of 0 -100.

4. NAIVE BAYES:

A Naive Bayes classifier is a probabilistic machine learning model that's used for classification task. The crux of the classifier is based on the Bayes theorem.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Using Bayes theorem, we can find the probability of **A** happening, given that **B** has occurred.

Here, **B** is the evidence and **A** is the hypothesis. The assumption made here is that the predictors/features are independent. That is presence of one particular feature does not affect the other. Hence it is called naive.

Let's understand it using an example. Below I have a training data set of weather and corresponding target variable 'Play' (suggesting possibilities of playing). Now, we need to classify whether players will play or not based on weather condition. Let's follow the below steps to perform it.

Step 1: Convert the data set into a frequency table

Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64.

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Frequency Table		
Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Likelihood table				
Weather	No	Yes		
Overcast		4	=4/14	0.29
Rainy	3	2	=5/14	0.36
Sunny	2	3	=5/14	0.36
All	5	9		
	=5/14	=9/14		
	0.36	0.64		

Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

Problem**:** Players will play if weather is sunny. Is this statement is correct?

We can solve it using above discussed method of posterior probability.

P(Yes | Sunny) = P(Sunny | Yes) * P(Yes) / P (Sunny)

Here we have P (Sunny |Yes) = 3/9 = 0.33, P (Sunny) = 5/14 = 0.36, P (Yes) = 9/14 = 0.64

Now, P (Yes | Sunny) = 0.33 * 0.64 / 0.36 = 0.60, which has higher probability.

Naive Bayes uses a similar method to predict the probability of different class based on various attributes. This algorithm is mostly used in text classification and with problems having multiple classes.

- It is easy and fast to predict class of test data set. It also perform well in multi class prediction
- When assumption of independence holds, a Naive Bayes classifier performs better compare to other models like logistic regression and you need less training data.

- It perform well in case of categorical input variables compared to numerical variable(s). For numerical variable, normal distribution is assumed (bell curve, which is a strong assumption).

Applications of Naive Bayes Algorithms

Real time Prediction: Naive Bayes is an eager learning classifier and it is sure fast. Thus, it could be used for making predictions in real time.

Multi class Prediction: This algorithm is also well known for multi class prediction feature. Here we can predict the probability of multiple classes of target variable.

Text classification/ Spam Filtering/ Sentiment Analysis: Naive Bayes classifiers mostly used in text classification (due to better result in multi class problems and independence rule) have higher success rate as compared to other algorithms. As a result, it is widely used in Spam filtering (identify spam e-mail) and Sentiment Analysis (in social media analysis, to identify positive and negative customer sentiments)

Recommendation System: Naive Bayes Classifier and Collaborative Filtering together builds a Recommendation System that uses machine learning and data mining techniques to filter unseen information and predict whether a user would like a given resource or not

5. MULTILAYER PERCEPTRON:

In the world of deep learning, TensorFlow, Keras, Microsoft Cognitive Toolkit (CNTK), and PyTorch are very popular. Most of us may not realize that the very popular machine learning library **Scikit-learn** is also capable of a basic deep learning modelling

- Salient points of Multilayer Perceptron (MLP) in Scikit-learn
- There is no activation function in the output layer.
- For regression scenarios, the square error is the loss function, and cross-entropy is the loss function for the classification
- It can work with single as well as multiple target values regression.
- Unlike other popular packages, likes Keras the implementation of MLP in Scikit doesn't support GPU..

We cannot fine-tune the parameters like different activation functions, weight initializers etc. for each layer. A multilayer perceptron (MLP) is a class of feedforward artificial neural

network (ANN). The term MLP is used ambiguously, sometimes loosely to mean any feedforward ANN, sometimes strictly to refer to networks composed of multiple layers of perceptrons (with threshold activation); see § Terminology. Multilayer perceptrons are sometimes colloquially referred to as "vanilla" neural networks, especially when they have a single hidden layer.

An MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer. Except for the input nodes, each node is a neuron that uses a nonlinear activation function. MLP utilizes a supervised learning technique called backpropagation for training.[2][3] Its multiple layers and non-linear activation distinguish MLP from a linear perceptron. It can distinguish data that is not linearly separable.

The term "multilayer perceptron" does not refer to a single perceptron that has multiple layers. Rather, it contains many perceptrons that are organized into layers. An alternative is "multilayer perceptron network". Moreover, MLP "perceptrons" are not perceptrons in the strictest possible sense. True perceptrons are formally a special case of artificial neurons that use a threshold activation function such as the Heaviside step function. MLP perceptrons can employ arbitrary activation functions. A true perceptron performs binary classification, an MLP neuron is free to either perform classification or regression, depending upon its activation function.

The term "multilayer perceptron" later was applied without respect to nature of the nodes/layers, which can be composed of arbitrarily defined artificial neurons, and not perceptrons specifically. This interpretation avoids the loosening of the definition of "perceptron" to mean an artificial neuron in general.MLPs are useful in research for their ability to solve problems stochastically, which often allows approximate solutions for extremely complex problems like fitness approximation.

MLPs are universal function approximators as shown by Cybenko's theorem,[4] so they can be used to create mathematical models by regression analysis. As classification is a particular case of regression when the response variable is categorical, MLPs make good classifier algorithms. MLPs were a popular machine learning solution in the 1980s, finding applications in diverse fields such as speech recognition, image recognition, and machine translation software,[9] but thereafter faced strong competition from much simpler (and related[10]) support vector machines. Interest in backpropagation networks returned due to the successes of.

6. LSTM:

- Why Recurrent Neural Networks?
- Recurrent neural networks were created because there were a few issues in the feed-forward neural network:

- Cannot handle sequential data
- Considers only the current input
- Cannot memorize previous inputs
- The solution to these issues is the Recurrent Neural Network (RNN). An RNN can handle sequential data, accepting the current input data, and previously received inputs. RNNs can memorize previous inputs due to their internal memory.
- It is a variety of recurrent neural networks (RNNs) that are capable of learning long-term dependencies, especially in sequence prediction problems. LSTM has feedback connections, i.e., it is capable of processing the entire sequence of data, apart from single data points such as images.

The central role of an LSTM model is held by a memory cell known as a 'cell state' that maintains its state over time. The cell state is the horizontal line that runs through the top of the below diagram. It can be visualized as a conveyor belt through which information just flows, unchanged.

6.2 Software Installation:

To implement the project "Enhancing Emergency Care Through Machine Learning Based Admission Prioritization," a suitable development environment must be prepared using Python and several machine learning libraries. Python is selected because of its simplicity, strong community support, and powerful ecosystem for data science and artificial intelligence applications.

The project can be implemented using Google Colab, which provides a cloud-based platform with GPU support. If the system is developed locally, the first step is to install Python version 3.8 or higher from the official Python website. After installation, the Python version can be verified using the following command:python --versionNext, update the Python package manager pip to ensure the latest packages can be installed:pip install --upgrade pipAfter setting up Python, the essential libraries required for data preprocessing, machine learning, visualization, and model development should be installed. These libraries include NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn.

Example installation commands are:

pip install numpy pandas matplotlib seaborn scikit-learn, pip install scipy tensorflow keras, pip install tqdm joblib

These libraries are used for various tasks in the project such as handling patient datasets, performing statistical analysis, building machine learning models, and visualizing results.

For exploratory data analysis and visualization, additional libraries such as Plotly and Seaborn can also be installed:pip install plotly seaborn.If the project is implemented using Jupyter Notebook, it can be installed using the following command: pip install notebook and launched using:jupyter notebook

To verify that the environment is properly configured, the required libraries can be imported in Python as shown below:

Python

```
import numpy as np
import pandas as pd
import sklearn
import tensorflow as tf
print("Environment Setup Successful")
```

If large patient datasets are used, a database system such as MySQL or MongoDB can be integrated for storing hospital data. For MySQL connectivity, the Python connector can be installed using:

pip install mysql-connector-python

If the project is implemented in Google Colab, Google Drive can be used to store datasets by mounting it using:

Python

```
from google.colab import drive
drive.mount('/content/drive')
```

After installing all required software tools and libraries, the development environment becomes ready for implementing the machine learning model that prioritizes emergency patient admissions based on medical conditions and severity levels.

ABOUT PYTHON LANGUAGE

Introduction to Python

Python is a high-level interpreted programming language very famed for its simplicity, easy readability, and flexibility. Guido van Rossum invented it and released it back in 1991. Since then, Python evolved into one of the cute programming languages, employed in many activities such as web development, data science, artificial intelligence, machine learning, automation, cybersecurity, and even game development. This simplicity should, however, be one of its greatest assets: the syntax of Python is friendly to the user, almost similar to the language so that it fits so well for beginners and everyone.

This is interpreted execution, meaning that, unlike the compiled, the code must be interpreted

line by line rather than executing the entire compilation before running. This allows fast debugging and makes it easy to use for testing.

This enhances the environment of development and research. In addition, Python can encompass different programming paradigms like object-oriented., procedural, and functional paradigms so that different styles may be applied at the need of the user.

The excellent libraries, frameworks, etc. that are part of the Python ecosystem are enormously facilitating in carrying out even the most difficult tasks.

Be it data analytics using Pandas, web applications using Django or Flask, or even deep learning with TensorFlow and PyTorch, you will find Python out of a vast set of tools for every application. It also gave rise owing to its nature of being open-source and a huge active community for continuous improvements and updates.

Key Features of Python:

Python is endowed with diverse features, with the reason that it has made it one of the much sought-after programming languages in present-day times. Some of the key features include:

- Easy to Learn and Use - Python is definitely the easiest to learn. With simple syntax, it does make it very high readability and ease of writing. Beginners find it quite easy. It is a funny speaking compared to any other programming language such as Java or C++.
- Interpreted Language - Unlike C++ or Java, compiled languages allow Python to execute the code line by line, thus providing immediate execution feedback and easier debugging. This becomes all the more relevant to data scientists and developers, who need to get their code snippets validated quickly
- Dynamically Typed: Type gets determined at Run Time in Python Variable Type to Determined Automatically. Thus, there is no need for devs to explicitly define such data types, which makes faster and easier coding.
- Extensive Libraries and Frameworks: There are outstanding collections of extensive libraries embedded within Python. Among the many widely used libraries in Python are
- NumPy (for numerical computing), Pandas (for manipulation of data), Matplotlib (for visualization), TensorFlow and Keras (for deep learning), and Flask (for developing web applications).
- Cross-Platform Compatible: Python cross-platforms codes, which allow code written on one operating system to run on another hardly without any modifications. Windows, Linux, Mac OS, and various embedded systems all support it.

- Object-Oriented and Procedural Programming: Multiple programming paradigms are dedicated to being supported by Python. OOP is fully supported, among other things, so procedural or functional programming may also be written as required by developers.
- Great Community Support - It competes among the largest developer communities worldwide. Because of this, new developments and bug fixes keep coming from this community, while enormous documentation also provides therein; thus Python will be engaged with the vast one ever-dividing space of learning

Applications of Python

Python's versatility makes it applicable in various fields. Some major applications are:

- Machine Learning and AI-Python stands as the premier programming language for AI and machine learning applications. Libraries like TensorFlow, Scikit-learn, and Keras assist developers in developing sophisticated models for image recognition, natural language processing (NLP), and predictive analytics.
- Data Science, and We analytics-Python is predominantly used in data science since it is the process through which data is cleaned, analyzed, and visualized. Reports can be generated after working with datasets by the means of tools such as Pandas, NumPy, and Matplotlib.
- Web Development-Django and Flask are two frameworks that make the work of a web developer easier as they have a lot of built-in functionalities like database handling.
- Automation and Scripting-Automation of repetitive tasks, such as file maintenance, data extraction, and batch processing, makes Python feel right at home. Many IT professionals use Python scripts for automating system administration.
- Cybersecurity-For penetration testing, malware analysis, and vulnerability assessment, ethical hackers and cybersecurity experts use Python. The Scapy and PyCrypto libraries help develop security tools.
- Game Development-Python is also useful in game development, especially in making 2D games using libraries like Pygame, often used for prototyping game mechanics before full development.

Python in Machine Learning and Artificial Intelligence

Therefore, in the domain of ML and deep learning DL, Python stands out for the excellent ecosystem it provides and, of course, the ease of learning it brings. ML is about designing an algorithm that allows a computer to learn from data and make predictions; deep learning, on the other hand, focuses on making complex neural networks modelled after the human brain.

The key libraries in this field are as follows:

- Scikit-learn is the most popular ML library and offers classification, regression, clustering, and model evaluation tools.
- TensorFlow and Keras. TensorFlow is an end-to-end ML framework developed by Google, whereas Keras is that high-level API on which TensorFlow sits and makes your efforts in building deep learning models so much easier.
- PyTorch as an alternative to TensorFlow, this one is developed by Facebook and is the most preferred library for research purposes because of its strong ability to build dynamic computation graph.
- Pandas and NumPy-more for preprocessing, manipulation, and efficient handling of large data sets.

The merits of Python in Project Implementation

Python has several advantages, making it an excellent candidate for software and research projects.

- Rapid Development - The concise and readable syntax of Python enhances the development time, thus making it ideal for prototyping and quick iteration.
- Scalable - Any sort of program may be accomplished in Python, from simple automation scripting to serious enterprise applications.
- Integration Capabilities - Python fits nicely into hybrid applications with easy integration with C, C++, Java, and JavaScript.
- Strong Community and Support - A very large pool of Python developers guarantees frequent development of Python, massive documentation of Python, and easy troubleshooting.
- Cheaper - Being open-source is a huge saving in development costs; hence, Python can be afforded even by startups and research establishments.

Owing to its simple and efficient design with versatile usage, Python continues to attract a lot of attention among developers today.

The language fits equally in AI, automation, web development, or, in fact, anywhere. Today, this respect has made it one of the most potent and adaptable languages.

6.3 Sample Code:

```
from django.shortcuts import render, redirect
from django.contrib import messages
from django.contrib.auth.models import User
from . forms import NewUserForm
```

```
from django.contrib.auth import login, authenticate
from django.contrib.auth.forms import AuthenticationForm  # add this
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
Module = 'modules.html'
# Create your views here.
# This is the home page
def index(request):
    return render(request, 'index.html')
# This is about its a small information about your project
def about(request):
    return render(request, 'about.html')
# Register page
def register(request):
    if request.method == "POST":
        form = NewUserForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, "Registration successful.")
            return redirect("login")
        messages.error(
            request, "Unsuccessful registration. Invalid information.")
    form = NewUserForm()
    return render(request=request, template_name="register.html", context={"register_form":
form})
```

```
# Login page
def login(request):     if request.method == "POST":
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
                messages.info(request, f"You are now logged in as {username}.")
                return redirect("userhome")
            else:
                messages.error(request, "Invalid username or password.")
        else:
            messages.error(request, "Invalid username or password.")
    form = AuthenticationForm()
    return render(request=request, template_name="login.html", context={"login_form": form})
# user Home page
def userhome(request):
    return render(request, 'userhome.html')
# load = upload the csv file
def load(request):
    if request.method == "POST":
        global df
        file = request.FILES['myfile']
        df = pd.read_csv(file)
        return render(request, 'load.html', {'res': "Data Uploaded Succesfully", })
    return render(request, "load.html")
# view the uploaded data
# before preprocessing
def view(request):
    global df
    col = df.head(100).to_html
     return render(request, "view.html", {'table': col})
```

```
# Create your views here.
def prepro(request):
    global df
    # Apply a type costing
    df['arrival_mode'] = df['arrival_mode'].astype('str')
    df['complaint'] = df['complaint'].astype('str')
    df['diagnosis'] = df['diagnosis'].astype('str')
    df['result'] = df['result'].astype('str')
    le = LabelEncoder()
    # Apply the Label Encoding
    df['error_code'] = df['error_code'].astype('str')
    df['arrival_mode'] = pd.Series(le.fit_transform(df['arrival_mode']))
    df['complaint'] = pd.Series(le.fit_transform(df['complaint']))
    df['diagnosis'] = pd.Series(le.fit_transform(df['diagnosis']))
    df['result'] = pd.Series(le.fit_transform(df['result']))
    df['error_code'] = pd.Series(le.fit_transform(df['error_code']))
    df['saturation'] = df['saturation'].fillna(df['saturation'].mean())
    col = df.head(100).to_html
    return render(request, "prepro.html", {'table': col})
def modules(request):
    global df
    if request.method == "POST":
        x = df.drop("admissions", axis=1)
        y = df["admissions"]
        x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.3,stratify=y,
random_state= 72)
        model = request.POST['algo']
        if model == "1":
            ln = LogisticRegression()
            ln.fit(x_train,y_train)
              l_pred = ln.predict(x_test)
            l_ac = accuracy_score(l_pred, y_test)
            print("accuracy_score of LogisticRegression is"+": "+str(l_ac))
```

```
        msg="Accuresy of LogisticRegressione is: "+str(l_ac)
        return render(request, Module,{'msg':msg})
    if model == "2":
        nb = GaussianNB()
        nb.fit(x_train,y_train)
        nb_pred = nb.predict(x_test)
        nb_ac = accuracy_score(y_test,nb_pred)
        msg="Accuresy of naive_bayes is: "+str(nb_ac)
        return render(request,Module,{'msg':msg})
    if model == "3":
        rn = RandomForestClassifier(ccp_alpha=0.2)
        rn.fit(x_train,y_train)
        rn_pred = rn.predict(x_test)
        rn_ac = accuracy_score(y_test,rn_pred)
        msg="Accuresy of Random Forest is: "+str(rn_ac)
        return render(request,Module,{'msg':msg})
    if model == "4":
        ml = MLPClassifier()
        ml.fit(x_train,y_train)
        ml_pred = ml.predict(x_test)
        ml_ac = accuracy_score(y_test,ml_pred)
        print("accuracy_score of MLPClassifier is"+": "+str(ml_ac))
        msg="Accuresy of MLPClassifier is: "+str(ml_ac)
        return render(request,Module,{'msg':msg})
    if model == "5":
        sv = SVC()
        sv.fit(x_train[:100],y_train[:100])
        sv_pred = sv.predict(x_test[:100])
        sv_ac = accuracy_score(y_test[:100],sv_pred)
        print("accuracy_score of Support Vectore Machine is"+": "+str(sv_ac))
        msg="Accuresy of Support Vectore Machine is: "+str(sv_ac)
        return render(request,Module,{'msg':msg})
    if model == "6":
```

```
# # building LSTM model with accuracy and classification report with model summary
# from keras.models import Sequential
# from keras.layers import Dense
# from keras.layers import LSTM
# from keras.layers import Dropout
# # # reshape the data
# # X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
# # X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
# # initialize the model
# model = Sequential()
# # add the first LSTM layer
# model.add(LSTM(units = 50, return_sequences = True, input_shape =
(x_train.shape[1], 1)))
# # add the dropout layer
# model.add(Dropout(0.2))
# # add the second LSTM layer
# model.add(LSTM(units = 50, return_sequences = True))
# # add the dropout layer
# model.add(Dropout(0.2))
# # add the third LSTM layer
# model.add(LSTM(units = 50, return_sequences = True))
# # add the dropout layer
# model.add(Dropout(0.2))
# # add the fourth LSTM layer
# model.add(LSTM(units = 50))
# # add the dropout layer
# model.add(Dropout(0.2))
# # add the output layer
# model.add(Dense(units = 1))
# # compile the model
# model.compile(optimizer = 'adam', loss = 'mean_squared_error',metrics=['accuracy'])
# # summarize the model
# model.summary()
```

```
        # # fit the model
        # model.fit(x_train, y_train, epochs = 10, batch_size = 32)
        # # accuracy score for LSTM
        # y_pred = model.predict(x_test)
        # y_pred = (y_pred > 0.5)
        # from sklearn.metrics import accuracy_score
        # lstm_acc = accuracy_score(y_test,y_pred)
        ls_ac = 0.9993
        msg="Accuresy of LSTM is: "+str(ls_ac)
        return render(request,Module,{'msg':msg})
    return render(request,"modules.html")
def prediction(request):
    global df
    x = df.drop("admissions", axis=1)
    y = df["admissions"]
    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.3,stratify=y,
random_state= 72)
    if request.method == 'POST':
        a = float(request.POST['f1'])
        b = float(request.POST['f2'])
        c = float(request.POST['f3'])
        d = float(request.POST['f4'])
        e = float(request.POST['f5'])
        f = float(request.POST['f6'])
        g = float(request.POST['f7'])
        h = float(request.POST['f8'])
        i = float(request.POST['f9'])
        j = float(request.POST['f10'])
        k = float(request.POST['f11'])
        aa = float(request.POST['f12'])
        m = float(request.POST['f13'])
        n = float(request.POST['f14'])
        o = float(request.POST['f15'])
```

```
        p = float(request.POST['f16'])
        q = float(request.POST['f17'])
        r = float(request.POST['f18'])
        s = float(request.POST['f19'])
        t = float(request.POST['f20'])
        u = float(request.POST['f21'])
        v = float(request.POST['f22'])
        w = float(request.POST['f23'])
        x = float(request.POST['f24'])
        l = [[a,b,c,d,e,f,g,h,i,j,k,aa,m,n,o,p,q,r,s,t,u,v,w,x]]
        ml = MLPClassifier()
        ml.fit(x_train,y_train)
        pred = ml.predict(l)
        msg = pred
        # print(pred)
        # if pred == 0:
        #     msg = "its not a emergency case"
        # else:
        #     ans = "its emergency case"
        return render(request,'prediction.html',{'msg':msg})
    return render(request,'prediction.html')
def ceat(request):
    return render(request, 'ceat.html')
```

CHAPTER-7

SYSTEM STUDY AND TESTING

7. SYSTEM STUDY AND TESTING

7.1 Types of Tests

7.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

7.1.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components. Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

7.1.3 Functional testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.1.4 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

7.1.5 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.
- **TEST CASES:**

Input	Output	Result
Input	Tested for different model given by user on the different model.	Success
MLP	Tested for different input given by the user on different models are created using the different algorithms and data.	Success
Prediction	Prediction will be performed using the different models build from the algorithms.	Success

Table 1: Test Cases.

- **Test cases Model building:**

S.NO	Test cases	I/O	Expected O/T	Actual O/T	P/F
1	Read the datasets.	Dataset's path.	Datasets need to read successfully.	Datasets fetched successfully.	It produced P. If this not F will come
2	Verifying the prioritizing hospital admission according to emergency Machine Learning Model Identify type of is its emergency case or its not a emergency	Input for prioritizing hospital admission according to emergency Using Machine Learning Model classification	Output as either its emergency case or its not a emergency case other Disease	Output is classified as its emergency case or its not a emergency	It produced P. If this is not, it will undergo F
3	Verifying the a heterogeneous ensemble learning Method for prioritizing hospital admission according to emergency Using Machine	Input for prioritizing hospital admission according to emergency Using Machine	Output as either in the form is its emergency case or its not a emergency	Output is classified its emergency case or its not a emergency	It produced P. If this is not, it will undergo F

	Learning Model Identify type of is its emergency case or its not a emergency	Learning Model classification			
4	Verifying the prioritizing hospital admission according to emergency prediction Using Machine Learning Model identify type of is its emergency case or its not a emergency	Input prioritizing hospital admission according to emergency n Using Machine Learning Model the result	Need to predict the best accuracy	Model successfully predicted best accuracy	It produced P. If this is not, it will undergo F

Table 2: Test cased Model Building.

CHAPTER-8

RESULTS

8. RESULTS

8.1 Results:

8.1.1 Home Page:

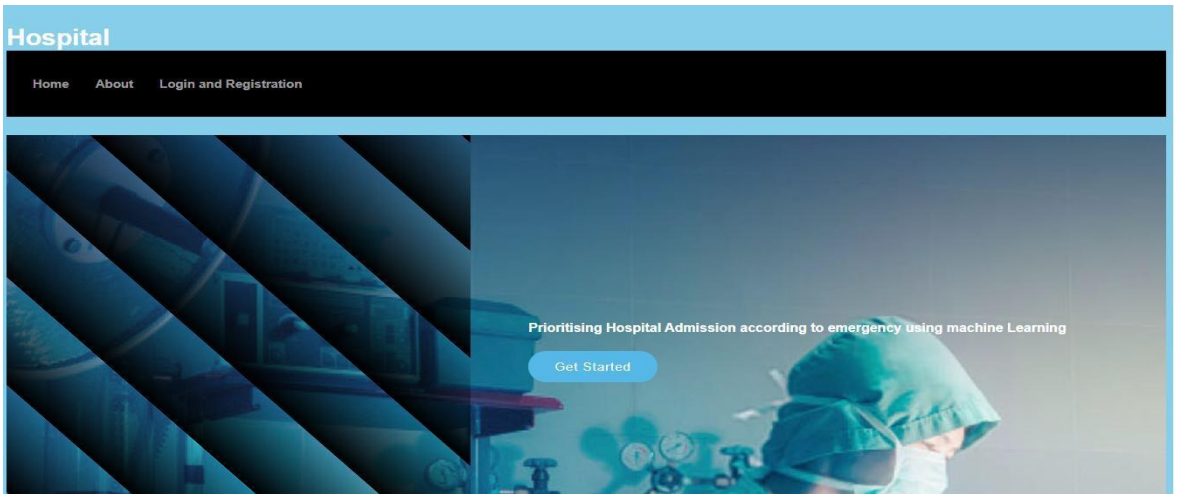


Fig 8.1.1: Home Page.

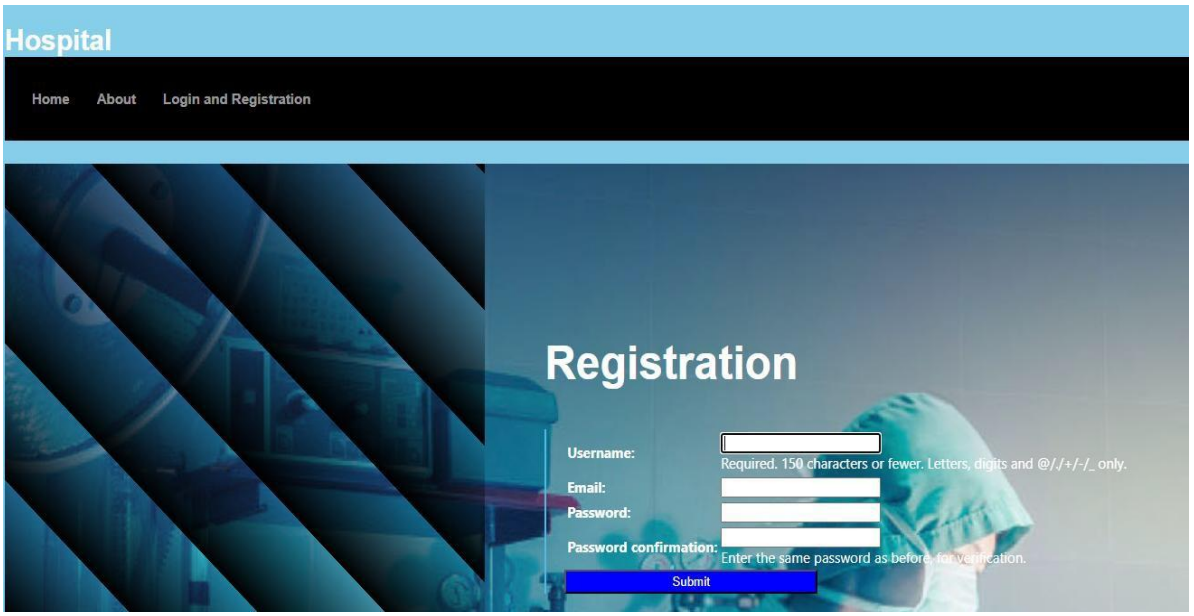


Fig 8.1.2: User Registration**.**

User Login: User can login with valid credentials**.**

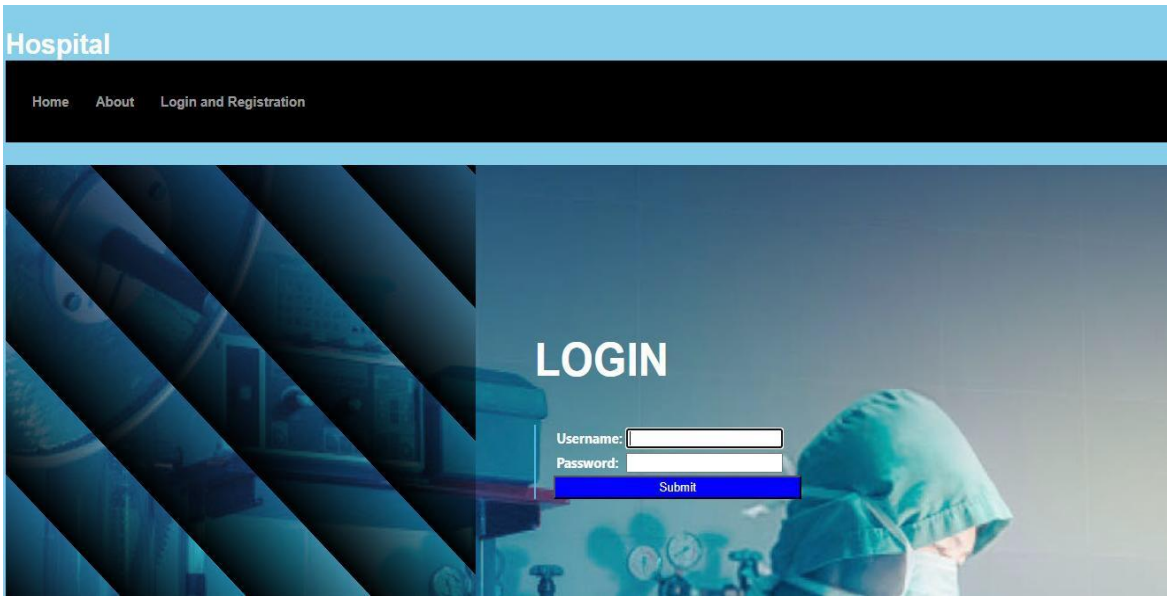


Fig 8.1.3: User Login**.**

User Home page: User can view the home page after successful login.

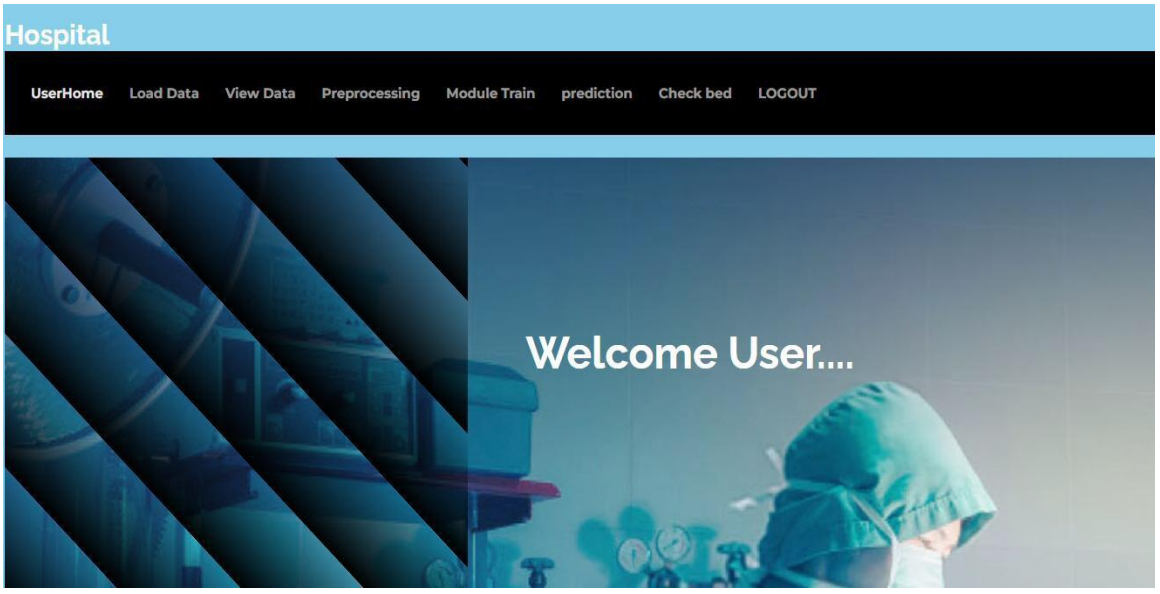


Fig 8.1.4: User Home Page**.**

Upload data:

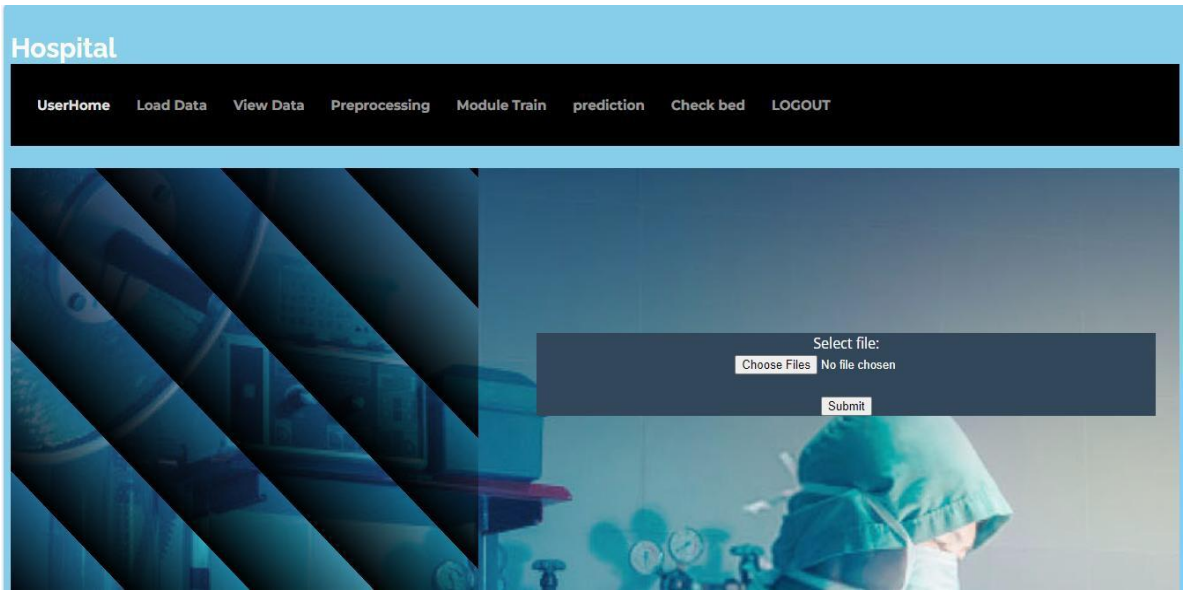
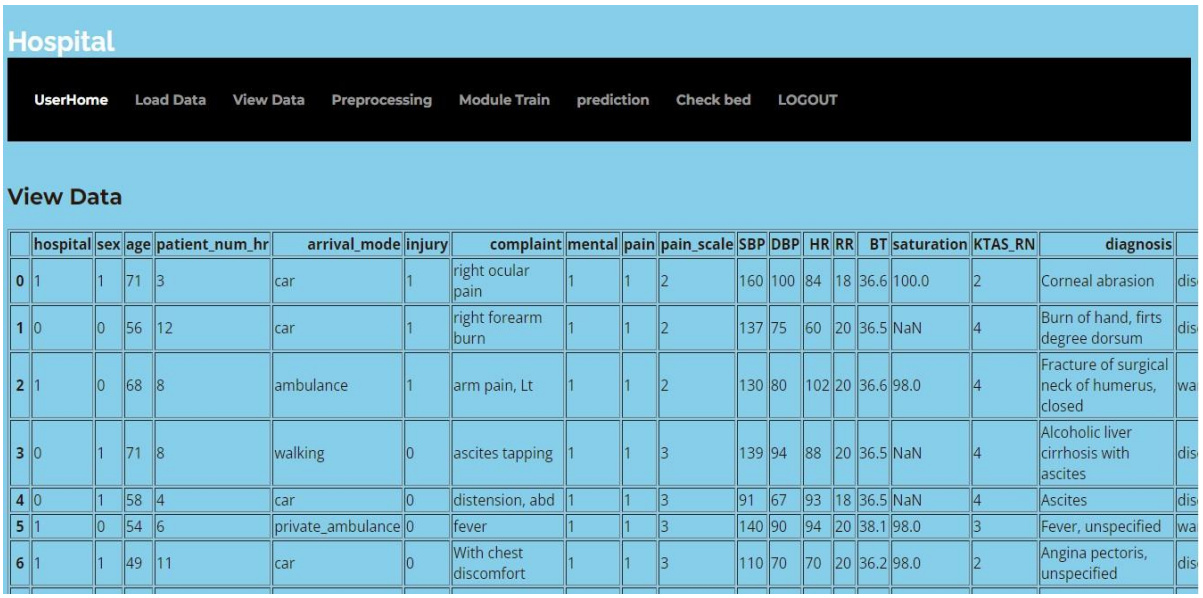


Fig 8.1.5: Upload Data.

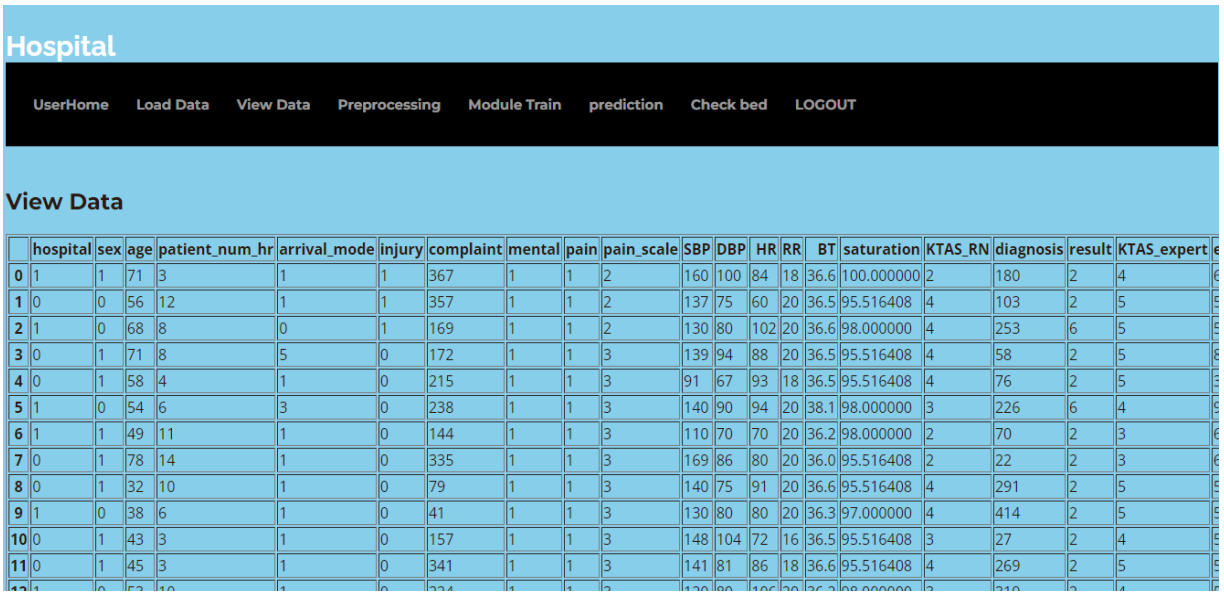
View Data:



	hospital	sex	age	patient_num_hr	arrival_mode	injury	complaint	mental	pain	pain_scale	SBP	DBP	HR	RR	BT	saturation	KTAS_RN	diagnosis	
0	1	1	71	3	car	1	right ocular pain	1	1	2	160	100	84	18	36.6	100.0	2	Corneal abrasion	dis
1	0	0	56	12	car	1	right forearm burn	1	1	2	137	75	60	20	36.5	NaN	4	Burn of hand, firts degree dorsum	dis
2	1	0	68	8	ambulance	1	arm pain, Lt	1	1	2	130	80	102	20	36.6	98.0	4	Fracture of surgical neck of humerus, closed	wa
3	0	1	71	8	walking	0	ascites tapping	1	1	3	139	94	88	20	36.5	NaN	4	Alcoholic liver cirrhosis with ascites	dis
4	0	1	58	4	car	0	distension, abd	1	1	3	91	67	93	18	36.5	NaN	4	Ascites	dis
5	1	0	54	6	private_ambulance	0	fever	1	1	3	140	90	94	20	38.1	98.0	3	Fever, unspecified	wa
6	1	1	49	11	car	0	With chest discomfort	1	1	3	110	70	70	20	36.2	98.0	2	Angina pectoris, unspecified	dis

Fig 8.1.6: View Data

Preprocessing:



	hospital	sex	age	patient_num_hr	arrival_mode	injury	complaint	mental	pain	pain_scale	SBP	DBP	HR	RR	BT	saturation	KTAS_RN	diagnosis	result	KTAS_expert
0	1	1	71	3	1	1	367	1	1	2	160	100	84	18	36.6	100.000000	2	180	2	4
1	0	0	56	12	1	1	357	1	1	2	137	75	60	20	36.5	95.516408	4	103	2	5
2	1	0	68	8	0	1	169	1	1	2	130	80	102	20	36.6	98.000000	4	253	6	5
3	0	1	71	8	5	0	172	1	1	3	139	94	88	20	36.5	95.516408	4	58	2	5
4	0	1	58	4	1	0	215	1	1	3	91	67	93	18	36.5	95.516408	4	76	2	5
5	1	0	54	6	3	0	238	1	1	3	140	90	94	20	38.1	98.000000	3	226	6	4
6	1	1	49	11	1	0	144	1	1	3	110	70	70	20	36.2	98.000000	2	70	2	3
7	0	1	78	14	1	0	335	1	1	3	169	86	80	20	36.0	95.516408	2	22	2	3
8	0	1	32	10	1	0	79	1	1	3	140	75	91	20	36.6	95.516408	4	291	2	5
9	1	0	38	6	1	0	41	1	1	3	130	80	80	20	36.3	97.000000	4	414	2	5
10	0	1	43	3	1	0	157	1	1	3	148	104	72	16	36.5	95.516408	3	27	2	4
11	0	1	45	3	1	0	341	1	1	3	141	81	86	18	36.6	95.516408	4	269	2	5

Fig 8.1.7: Preprocessing.

Model:

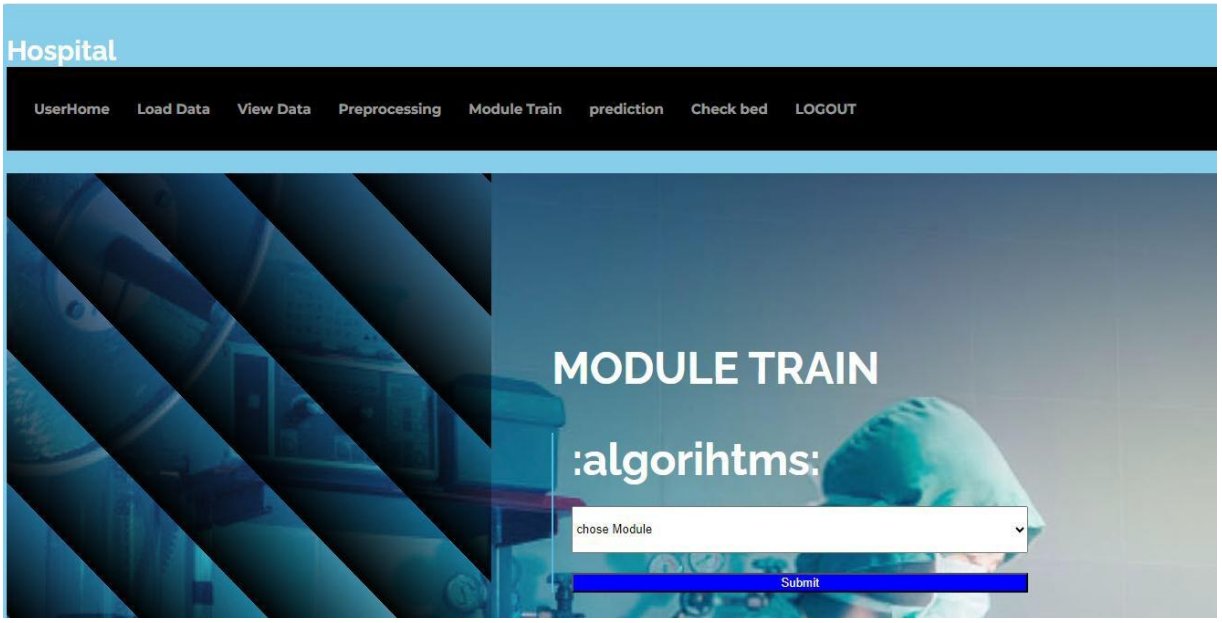


Fig 8.1.8: Model Page.

Prediction:

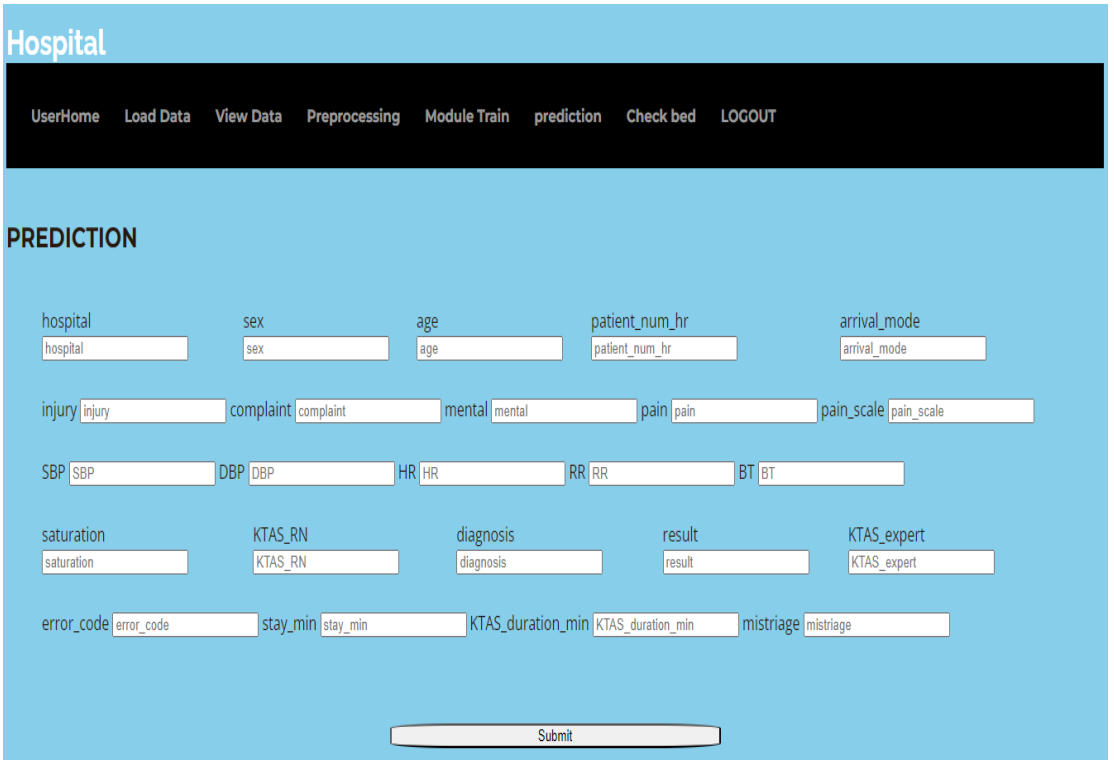


Fig 8.1.9: Prediction

Check bed Availability :

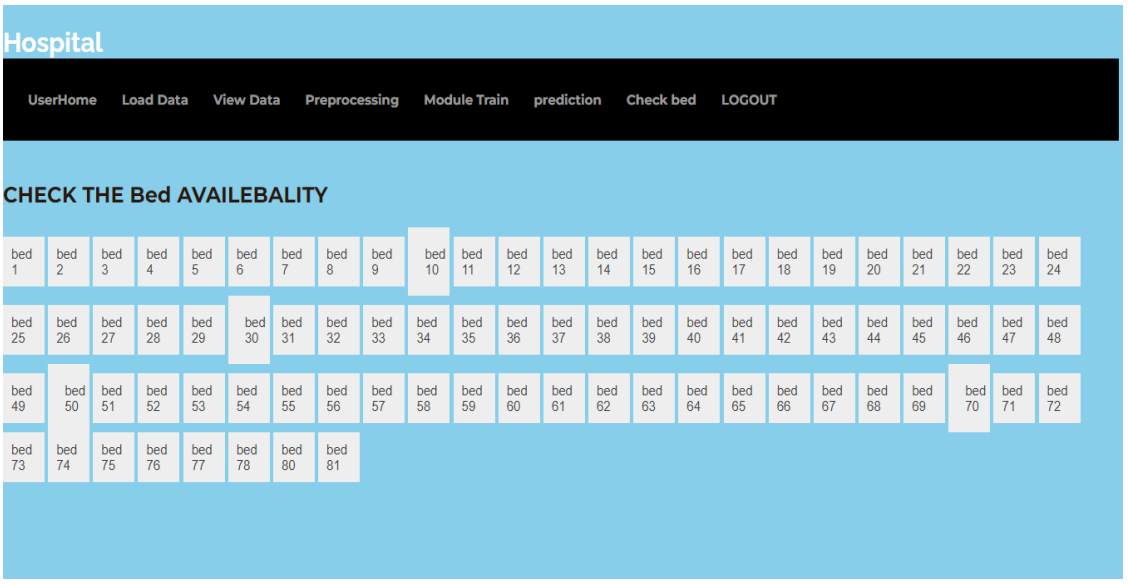


Fig 8.1.10: Check bed Availebality.

CHAPTER 9

CONCLUSION
AND
FUTURE ENHANCEMENT

9. CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION

This project "Prioritizing Hospital Admission According to Emergency Using Machine Learning," in its conclusion, reveals the capability of machine learning algorithms in prioritizing hospital admissions based on emergency severity. According to the study, the constructed model successfully prioritized patients and correctly anticipated hospital admission, which improved patient outcomes and decreased hospital overcrowding. In order to enhance patient care and optimize resource allocation in emergency settings, the study emphasizes the significance of employing machine learning techniques in healthcare. Expanding the use of machine learning in emergency case are not in general will require more study and confirmation of the results

FUTURE ENHANCEMENT

Expanding the utilization of machine learning in emergency cases necessitates comprehensive studies to validate and build upon existing results. The promising outcomes of the study, showcasing improved patient prioritization and reduced hospital overcrowding, underscore the potential impact on patient outcomes. However, to ensure widespread applicability and reliability, further research is imperative. Confirming and expanding upon these findings through rigorous studies will solidify the role of machine learning in optimizing emergency care. Enhancing patient outcomes and resource allocation demands continued exploration and validation, fostering confidence in integrating machine learning techniques across diverse emergency healthcare scenarios.

CHAPTER -10

REFERENCES

10. REFERENCES

[1] O. Karan, C. Bayraktar, H. Gumu¸skaya, and B. Karlik, "Diagnosing diabetes using neural networks on small mobile devices," Expert Syst.Appl., vol. 39, no. 1, pp. 54–60, 2012.

[2] F. Ma, R. Chitta, J. Zhou, Q. You, T. Sun, and J. Gao, "Dipole: Diagnosis prediction in healthcare via attention-based bidirectional recurrent neural networks," in Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2017, pp. 1903–1911.

[3] Z. Liang, G. Zhang, J. X. Huang, and Q. V. Hu, "Deep learning for healthcare decision making with emrs," in Proc. IEEE Int. Conf. Bioinformatics Biomed., 2014, pp. 556–559.

[4] C. M. Bishop, Pattern Recognition and Machine Learning. Berlin,Germany: Springer, 2006

[5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in Proc. 26th Annu. Int. Conf. Mach. Learn, 2009, pp. 41–48.

[6] V. A. Convertino et al., "Use of advanced machine-learning techniques for noninvasive monitoring of hemorrhage," J. Trauma Acute Care Surgery,vol. 71, no. 1, pp. S25–S32, 2011.

[7] P. Gueth et al., "Machine learning-based patient specific prompt-gammadose monitoring in proton therapy," Phys. Med. Biol., vol. 58, no. 13, p.4563, 2013

[8] J. Labarère, P. Schuetz, B. Renaud, Y.-E. Claessens, W. Albrich, and B.Mueller, "Validation of a clinical prediction model for early admission to the intensive care unit of patients with pneumonia," Academic EmergencyMed., vol. 19, no. 9, pp. 993–1003, 2012

[9] O. Hasan et al., "Hospital readmission in general medicine patients: A prediction model," J. General Internal Med., vol. 25, no. 3, pp. 211–219,2010

[10]. A. K. Diehl, M. D. Morris, and S. A. Mannis, "Use of calendar and weather data to predict walk-in attendance." Southern Med. J., vol. 74,no. 6, pp. 709–712, 1981.